SAND WORKS

Product & Technical Specification
and Implementation Control Plane

Owner: Ramesh Sahu · Package: com.roshan.sandworks
Online-first · Roles OWNER / DRIVER / LABOURER (no ADMIN)

Documentation only — no application source. Revision v0.2.0 (2026-09-08).

Contents

Status classification: EXISTING / SPECIFIED / MISSING / BLOCKED / PROPOSED. Specification is NOT implementation.

README

SAND WORKS

SAND WORKS — an online-first Android application for a small farming/labour operation. It is a **private, personal/family operational tool** for a single owner who runs tractors, drivers and daily labourers, records trips, and transparently shows each person what they have **accrued**.

This repository is the **single authoritative product + technical specification and implementation control plane**. It is documentation, architecture, requirements, UX, security, planning and validation only — **it contains no application source code**. An independent senior Android (Kotlin + Jetpack Compose) coding agent builds the app from this repository, task-by-task, per `docs/implementation/`.

Status

• **Direction:** online-first. Cloud Firestore/Firebase is the authoritative backend. Not offline-first, and not derived from any prior/legacy application.

• **Scope:** private, one-owner, family/operator use. Not a public SaaS, marketplace, or social product. Not slated for Google Play unless the owner later changes that.

• **Readiness:** SPECIFIED (build-ready) — see `docs/DOCUMENTATION-INDEX.md` for the full status classification of every area.

Identity

Roles (summary)

• **OWNER (Ramesh Sahu):** complete control within the documented boundary — users/approvals, tractors, trips, rate & distribution config, accruals, daily closure, attendance, leaderboards, notifications, emergency alerts, messaging, exports, audit.

• **DRIVER (e.g. Mansingh Rana):** authorized driver operations — dashboard, tractor selection, add/edit trips, select labourers, own history/totals, share today's trips, temporary-assignment support when authorized. Never an admin.

• **LABOURER:** operationally **read-only** — own accrued money, working/absent days/dates, weekly + monthly top-3 leaderboard, profile, notifications.

What this repository holds

Read order

1. `AGENT.md` — the binding rules.
2. `docs/DOCUMENTATION-INDEX.md` — full index + status classification.
3. `PRD.md` → `PRD2.md` — what and how in detail.
4. `docs/` — screens, navigation, design system, database, API, security, money, notifications, testing, deployment.
5. `docs/implementation/` — how to build, task by task.

No legacy/reference-application material is included. This is a fresh, self-contained specification.

AGENT.md — SAND WORKS AI Coding-Agent Constitution

This file is the **binding constitution** for every future AI coding agent (or human contributor) working on SAND WORKS. It is deliberately strict. If you are an AI agent, treat every "must" below as mandatory. When anything is genuinely not implementable, **STOP at that boundary and report the exact blocker** — do not improvise.

1. Authority

Follow the authoritative documentation. Where documents conflict, the resolution order is:

1. `AGENT.md` (this file)
2. `README.md`
3. `PRD.md` and `PRD2.md`
4. `docs/DOCUMENTATION-INDEX.md` (index/status)
5. The specific `docs/` specification documents
6. `docs/implementation/` task contracts

Nothing else is authoritative. Older or undocumented material is never a source of truth.

2. Never fabricate (absolute prohibitions)

Never use or introduce:

• mock/dummy/fake data, fabricated backend or API responses

• placeholder production logic, UI, logos, icons, or images

• fake loading completion, fake success messages, fake progress

• hard-coded fake users, money, trips, roles, or accruals

• fake Firebase configuration, fake credentials, fake notification delivery

• fake authentication or fake authorization

• fake payment/accrual results

• a fake implementation solely to make a build pass

Never leave in production code or documentation:

• `TODO`, `FIXME`, `TBD`, `XXX`, `HACK`, "implement later"

• incomplete production branches, commented-out unfinished code

• empty production functions

• fake repository or network implementations

3. Business rules are fixed

Never invent, silently change, or reinterpret SAND WORKS business rules. Key invariants you must always honour:

• Exactly one OWNER (Ramesh Sahu). **No ADMIN role. Mansingh Rana is a DRIVER — never create a second administrator.**

• Money is **accrued-money** in **integer paise**; never floating point for authoritative money.

• Money messaging is a **daily accrued-money summary** — never label as "payment"/"paid".

• Rate default ■200 with **immutable per-trip rate snapshot**; changing the rate never rewrites history.

• Default distribution: equal split among eligible driver + participating labourers present; config only as documented.

• Backend is authoritative for money, trip numbering, permissions, closure, leaderboards, expiry.

• Labourers are operationally read-only.

• New driver/labourer accounts get **no privileged access until owner approval**, enforced by backend, never a fake local approval.

• Online-first architecture. Do not turn it into an offline-primary system; network failures use graceful UX, never fake success.

4. Security & integrity

• Never bypass security, weaken validation, or rely on UI hiding as a security control.

• Never suppress errors merely to make a screen look functional.

• Never disable tests to make CI green.

• Do not bypass or mock Firestore Security Rules, App Check, or Cloud Functions authorization.

• Deep links and cross-role access must be re-validated server-side.

• No cross-user private financial disclosure (no broadcasting one user's money to others).

5. Secrets & signing

• Never commit secrets, signing keys/keystores, or Firebase private credentials.

• Never put tokens in source code, markdown, `.env`, config, docs, or commit messages.

• Never use production credentials in tests.

• Use secure environment/CI secret injection only.

• Debug builds use standard debug signing; the private release uses a **dedicated new SAND WORKS keystore**, never a legacy key.

6. Verification before claiming completion

• Never claim completion without verification.

• Always inspect the existing implementation/documentation before modifying it.

• Maintain traceability between requirement → feature → screen → workflow → data → backend → security → test → implementation task → acceptance criteria.

• Update documentation whenever intended behaviour changes.

7. Assets & branding

• Use the approved PNG assets only. Do not regenerate, redraw, or create a replacement SAND WORKS logo.

• Do not create an SVG version of the approved logo unless explicitly requested.

• Masters are 1536×1536 (see `docs/brandreport/assets.md`). Do not choose canonical masters by guesswork.

8. Status honesty

Use the classification vocabulary exactly:

• **EXISTING** — actually present/implemented.

• **SPECIFIED** — required and fully documented, not yet implemented.

• **MISSING** — required but not yet specified enough.

• **BLOCKED** — cannot proceed until an external dependency/decision exists.

• **PROPOSED** — a marked recommendation not yet approved.

Never present PROPOSED as approved, SPECIFIED as IMPLEMENTED, a mock as production, or a planned backend operation as deployed.

9. Blocker protocol

If a task needs information or infrastructure that is genuinely absent (e.g. Firebase project, paid plan decision, signing key, an owner decision), **do not fabricate a substitute**. Mark the task BLOCKED in `docs/implementation/`, record it in the blocker register, and report the exact smallest input needed.

PRD — Product Requirements Document: SAND WORKS

Status: **SPECIFIED.** Revision 1.0. Owner-input items are marked clearly.

1. Product vision

A trustworthy, private, mobile operational tool that lets a single farm/operation owner coordinate tractors, drivers and daily labourers, record trips, and transparently communicate to each person exactly what they have **accrued** — ending guesswork and disputes over who worked and what they have earned.

2. Problem

A small operation hires drivers and daily labourers across several tractor trips each day. Manual tracking leads to:

• disputes over who worked on which trip/tractor and on which days;

• unclear money accrual (rate, distribution, totals);

• no single shared source of truth accessible to each person.

3. Users

4. Roles & goals

• **OWNER goals:** run the operation; approve/register users; manage tractors, trips, rates, distribution; review accruals, attendance, leaderboards; send messages and emergency warnings; export; audit.

• **DRIVER goals:** record trips accurately; pick tractor and labourers; see own totals/history; share today's trip count.

• **LABOURER goals:** see own accrued money, working/absent days and dates, weekly + monthly rank; stay informed via notifications.

5. Non-goals

• Not a public SaaS, marketplace, or social product.

• Not a general payroll/payment/disbursement system — accrual tracking only.

• No multi-owner, ADMIN, or delegated administrative role.

• No legacy data migration; not derived from any prior app.

• Not offline-primary; not a public-store release (unless owner later decides).

6. Functional requirements (summary — full detail in PRD2 and docs/)

Cover: auth lifecycle, owner/approval, tractor registry, trips, rate & distribution config, accrual & daily accrued-money closure (19:30 Asia/Kolkata default), attendance, leaderboards, notifications, emergency warning, messaging, exports, profile/photo, search/filter/sort, audit. See `PRD2.md` and the `docs/screens`, `docs/spec`, and `docs/architecture` documents for the authoritative per-feature detail.

7. Non-functional requirements

• **Online-first**, authoritative Firebase backend; graceful network UX.

• Android native (Kotlin + Compose + Material 3); package `com.roshan.sandworks`.

• Money in **integer paise**; never float for authoritative money.

• APK target ~15–25 MB (≤50 MB practical upper bound).

• Accessible (see `docs/quality/ACCESSIBILITY.md`), responsive, fast (see `PERFORMANCE.md`), secure (see `SECURITY.md`), testable (see `TESTING.md`).

• Private/family data handling; no analytics PII; approvals backend-enforced.

8. Business rules (invariants)

See `docs/spec/money-engine.md`, `docs/spec/roles-access.md`, `docs/architecture/DATABASE.md`, `API.md`, `SECURITY.md`. Key invariants are summarised in `AGENT.md` §3. Highlights: exactly one owner; no admin (Mansingh Rana = DRIVER); accrued-money wording ("daily accrued-money summary"); ■200 default rate with immutable per-trip snapshot; equal-split default distribution; backend-authoritative number/closure/leaderboard/expiry; read-only labourer; approval-before-access.

9. Acceptance criteria (top-level)

1. Owner can register/approve users and manage tractors, rate, distribution.
2. Authorized driver can create/edit trips with authoritative numbering and immutable rate snapshot.
3. Daily accrued-money closure runs idempotently at the configured evening time and never double-counts.
4. Each user sees only their own private figures; labourers are read-only.
5. Labourers see working/absent days/dates and honest weekly/monthly top-3 leaderboards (no fabricated ranks).
6. Owner can message and send emergency warnings; delivery is backend-authoritative and honest about Android limits.
7. All money is accrued-money in integer paise; never labelled "paid".
8. No fabricated data, backend, or notification anywhere; no secrets committed.

10. Dependencies & risks

• **External/blocked:** real Firebase project (Auth, Firestore, App Check, FCM, Crashlytics), paid-plan features (Storage/Cloud Functions where required) pending owner plan decision, release signing key. See `docs/implementation/status.md` and the blocker register.

• **Risks:** money double-counting (mitigated by idempotent closure + revision), cross-user money leakage (RBAC/rules), fake-offline drift (guarded by online-first policy), permission mis-configuration.

11. Privacy & security

Private/family data; least privilege; owner-only sensitive operations; backend authority; auditability; no PII analytics; secure Android storage; dedicated release signing; secrets never committed. Detail in `SECURITY.md` and `docs/architecture/DEPLOYMENT.md`.

12. Operational constraints

• Private APK distribution is the delivery target.

• Runs against an owner-provisioned Firebase backend.

• Must be maintainable by a small team of coding agents using `docs/implementation/`.

PRD2 — Execution-Level Product Specification: SAND WORKS

Status: **SPECIFIED.** This document adds implementation-useful precision on top of `PRD.md`. It does not duplicate `PRD.md`; it expands every feature, workflow, edge case, rule, validation, state, permission and backend interaction. Screens are specified in detail in `docs/screens/`; this file is the horizontal, cross-cutting product spec.

1. Identity, roles & routing (canonical)

• Owner = Ramesh Sahu (uid provisioned as OWNER). Exactly one.

• Mansingh Rana = DRIVER (primary/authorized). No admin/delegate role exists.

• Role routing table (after sign-in, role from backend session):

• OWNER → Owner dashboard.

• DRIVER → Driver dashboard.

• LABOURER → Labourer dashboard.

• pending approval → Approval Pending screen (no operational access).

• disabled/suspended/expired → Session Blocked → sign in.

2. Authentication & account lifecycle (states)

State machine for an account: `provisioned → pending → active | rejected | disabled | suspended`. Plus `session expired`, `logged out`.

• Sign Up: driver/labourer only for self-registration; owner account is provisioned by the operator (no public "create owner").

• New driver/labourer lands in `pending`; the OWNER approves or rejects in the Approvals screen. Until `active`, no operational data is returned (backend enforced).

• Password reset via backend secure flow. Session expiration and disabled/suspended handled truthfully; no fake success.

• Rejected user: sees rejection; may be re-registered/retried per owner.

• Approvals are recorded in audit.

3. Roles: detailed permission (capability) matrix

Read/create/update/delete/approve/configure/export/message/share per role. This is enforced by backend Security Rules + Cloud Functions, never UI-only. The authoritative matrix is in `docs/spec/roles-access.md` and `docs/architecture/SECURITY.md`. Representative rules:

• Only OWNER may: approve/reject users, disable/suspend, manage tractors (add/edit/deactivate/reactivate), set rate & distribution rule, correct attendance, export, send broadcasts/emergency warnings, read audit, see organisation-wide money.

• DRIVER may: manage own profile, read own dashboard/history/totals, add trips, edit own permitted trips within allowed window, select tractor from active registry and labourers, share today's trip count, support temporary assignment when authorized. DRIVER may NOT approve, configure money, export, see others' private money, or act as admin.

• LABOURER (read-only): own accrued money, working/absent days & dates, own trip participation, top-3 leaderboards + own rank, own profile, notifications. No writes to operational data.

4. Money & accrual precision

• Integer paise everywhere authoritative. ■ default 200 (=20000 paise) per trip, owner-configurable.

• Per-trip immutable **rate snapshot** at record time. Historical trips never rewritten by rate changes.

• Distribution rule (owner-configurable, exact allowed set): equal (default) among driver + eligible labourers present; document permitted alternatives (equal/driver+labour share/custom %/fixed). Rounding in paise: leftover paise assigned by a deterministic documented rule so totals reconcile exactly to the trip pool.

• Cases: zero labourers → pool to driver per rule; one labourer → split driver+one; multiple → equal/split per rule. Duplicate submission prevented by idempotency key + client in-flight guard. Edits allowed per owner/driver rule with revision-based conflict handling. Deletion/voiding documented (soft-void + audit, never hard delete of history). Historical-rate: never recomputed. Concurrency: revision on mutable docs; on conflict show user and choose. Retry: idempotency keys so retries never double-count.

• **Daily accrued-money summary / closure**: around configured evening schedule (recommended default **19:30 Asia/Kolkata**), server-side scheduled execution where required; idempotent boundary (organisation,date); exactly-once; never double-counts; wording = "accrued", never "payment". Detail: `docs/spec/money-engine.md`.

5. Tractor model

Owner-controlled registry, initially seeded with Sonalika and John Deere as owner-entered records (not hardcoded as the permanent complete set). Support ~4–5 operating tractors + multiple drivers.

Operations: add, edit, deactivate (soft), restore/reactivate; fields name, optional identifier, active/inactive; validation (unique name within org), duplicate handling; per-tractor totals; trip history; driver association where applicable. Detail: `docs/spec/tractors.md`.

6. Temporary labourer access (business-critical)

If the regular driver is absent, the OWNER (or authorized driver, within documented limits) temporarily assigns a specific labourer the access to perform driver duties. Requirements:

• target a specific user; explicit `start` and `expiry`; backend-enforced; automatically invalid after expiry; does not permanently change role; auditable; revocable by OWNER.

• Behaviour after expiry login → access denied truthfully (Session/Assignment Expired).

• Overlapping assignment → defined resolution (newest valid / reject overlap; documented).

• No unauthorized privilege escalation.

Specified as a state machine + security rule in `docs/spec/temporary-access.md`.

7. Notifications & emergency warning (honest Android)

• Normal notifications: account approval, access changes, trip events, **daily accrued-money summary** (never " payment"), operational updates, system/account events.

• Owner-only broadcast. Emergency warning: confirmation step, optional message, recipient scope, delivery state, retry, duplicate prevention, notification history, audit, cancellation where possible.

• Use the **strongest Android-compliant urgent notification behavior**: high-importance channel, sound, vibration, heads-up where the OS permits, full-screen intent only where platform-appropriate. **Never claim** the app can force full volume in Silent/DND, override volume, or bypass DND. Distinguish device volume vs Silent vs DND vs permission/policy restrictions. Detail: `docs/architecture/notifications.md`.

8. Profile & profile picture

Every role has a profile. Profile picture support: upload/replace/delete, image validation, size limits, compression, crop, loading, progress, failure, retry, authorization, privacy, storage lifecycle. Firebase Storage is **conditional on the Firebase plan**; if Storage needs a paid plan it is documented as a plan/environment dependency, never assumed present. Detail: `docs/spec/profile-photo.md`.

9. Search, filter, sort

Defined wherever useful: users, tractors, trips, dates, driver, labourer, status (active/inactive), working/absent, leaderboard period, money/accrual history. For each: input behaviour, debounce, case handling, empty state, reset, sort stability, pagination if required, loading/error, permissions, backend query implications. Detail:

`docs/spec/search-filter-sort.md`.

10. Attendance

Per labourer per date working/absent; owner marks and corrects with required reason; corrections audited; no silent overwrite; revision/conflict handling. Detail: `docs/spec/attendance.md`.

11. Leaderboards (honest)

Weekly leaderboard resets each week; monthly shows positions 1–3 only. Never fabricate ranks: if one qualifies show 1st only; if two show 1st–2nd only. Tie-break deterministic and documented. Minimum-data behaviour explicit. Timezone/period boundaries explicit (Asia/Kolkata). Detail: `docs/spec/leaderboards.md`.

12. Exports

Owner-only PDF + CSV, date-range and breakdown (overall/per tractor/per driver/per labourer/per date). Authorization by backend; generation; download/share per platform; export history; failure/retry. Detail: `docs/architecture/API.md`, `docs/spec/exports.md`.

13. Owner SOP

Operational procedures for the owner (approve, reject/block, add/deactivate tractor, add/edit trip, correct trip, configure rate, inspect accrual/leaderboards, send warning/message, export, manage profile, recover account, handle network/notification failure, temporary driver access, absent driver, review audit, prepare private APK release). Detail: `docs/spec/SOP.md`.

14. Edge cases & states (enumerated by operation)

Every operation in the system documents: success, validation error, permission denied, network failure, timeout, rate limited, conflict (concurrent edit), duplicate request, expired session, expired temporary access, feature unavailable (plan), server error, offline-with-retry. Consolidated in `docs/spec/ERROR-STATES.md`.

Product Overview — SAND WORKS

One-line

SAND WORKS is an **online-first** mobile tool that lets an owner operate tractors with drivers and daily labourers, record trips, and transparently show each person what they have accrued.

The problem it solves

A small farm/operation hires one or more drivers and daily labourers. Every day several tractor trips happen. Two things are hard to keep honest and transparent by hand:

1. **Who worked, on which trips, with which tractor**, and how many trips/days each person accumulated.

2. **What each person has accrued** in money terms, given a per-trip rate and a distribution rule among the driver and the labourers present on that trip.

SAND WORKS removes the guesswork and the arguments: the data lives in one shared online source of truth, and each person only ever sees their own figures.

Guiding principles (this product)

• **Online-first.** The authoritative data lives in a backend (Firebase). Devices read and write against the live backend; there is no local-primary/offline-primary data model. (Resilience against transient network issues is handled by normal loading/error/retry UI, not by an offline-first architecture.)

• **One organisation, one owner.** It is a private family/operator tool, not a multi-tenant SaaS and not a public marketplace.

• **Honest money.** The app tracks **accrued totals only** — never "payment completed". Wording, leaderboards and reports all reflect accrual, never disbursement.

• **Role-scoped.** A driver or labourer sees only their own operational information. Only the owner sees the full organisation.

• **No fabrication.** No fake data, fake success, or made-up backend behaviour anywhere.

Not in scope (explicit)

• No public/Play-store consumer release.

• No multi-owner or admin/delegated role.

• No disbursement/ledger/payment engine.

• No offline-first local database as the source of truth.

• No migration of data from any legacy application.

Key concepts

Online-first architecture in plain terms

• A person signs in with an account. New driver/labourer accounts need **owner approval** before privileged access.

• Reads and writes go to the live backend, protected by per-role security rules.

• The daily closure, trip numbering, money distribution and leaderboards are computed/authorised **by the backend**, never trusted from the client.

• See `docs/architecture/architecture.md` for the full model.

Roles — SAND WORKS

SAND WORKS has exactly **three roles**. There is **no** ADMIN/delegate role and exactly **one OWNER**.

People

• **OWNER: Ramesh Sahu** — the single operator and ultimate authority.

• **DRIVER: Mansingh Rana** — the primary/authorized driver, plus any additional approved drivers.

• **LABOURER** — daily workers, approved by the owner.

Important: *Mansingh Rana is a* ***DRIVER****. He is not an admin and SAND WORKS never creates a second administrative role. The architecture must not silently elevate him. As DRIVER he can perform the driver operations below; the OWNER remains the ultimate authority for approvals, configuration, money rules, exports and messaging.*

OWNER

Full visibility and control within the documented product boundary (no arbitrary undocumented powers — every capability has validation, authorization, error handling and audit):

• Dashboard, daily/historical summaries.

• Register/approve/reject/disable/suspend drivers & labourers; manage temporary access.

• Tractor registry (add/edit/deactivate/reactivate).

• Trips and trip corrections.

• Rate configuration and distribution-rule configuration.

• Money/accrual overview; per-person accrual detail; daily closure.

• Attendance (with reason for corrections).

• Leaderboards; notifications; emergency warnings; messages (owner-only broadcast).

• Exports (PDF/CSV, owner-only).

• Profile, application settings, security/account settings, audit/activity history.

DRIVER

An authorized driver (e.g. Mansingh Rana) can:

• Sign in; view dashboard and today's trips.

• View assigned/available tractors per authorization; select tractor; add trips; edit trips within allowed scope/window; select participating labourers.

• View relevant trip history; own accrued totals; operational summaries.

• Share the current day's total trip count (Android/WhatsApp share, real values).

• Manage profile; perform temporary-assignment operations only when authorized.

What a DRIVER cannot do: approve/reject users, configure rate or distribution, disable/suspend, export, send broadcasts, read another user's private financial information, or act as an admin. The DRIVER never becomes an OWNER or ADMIN.

LABOURER

Operationally **read-only**. Sees only their own data:

• Total trips, accrued money, remaining/accrued outstanding (as defined), working days, absent days, working dates, absent dates.

• Weekly leaderboard and monthly top-3 (honest), own rank, relevant trip participation.

• Profile, notifications, account/access status.

What a LABOURER cannot do: create/edit trips, change records, approve, configure, export, or see others' private money.

Approval model

• New DRIVER/LABOURER accounts are created as **pending** and gain **no privileged access** until the OWNER approves them (backend-enforced, never a fake local-only approval).

• Rejected/blocked, suspended, and disabled states all prevent operational access.

Temporary assignment

A LABOURER may, only while under a valid owner/authorized-driver temporary assignment, perform the assigned driver duties; the assignment has explicit start/expiry, is backend-enforced, expires automatically, never permanently changes role, and is revocable by the owner. See `docs/spec/temporary-access.md`.

Feature Catalogue — SAND WORKS

Requirement identifiers (SWF). Every feature is mapped to the screens and rules elsewhere in this repository. Nothing listed here is invented backend behaviour.

Identity & access

Owner operations

Driver operations

Labourer operations

Shared / cross-cutting

Out of scope

- Public/Play-store consumer release.
- Admin/delegate role; multiple owners.
- Payment disbursement / "paid" ledger.
- Legacy data migration.
- Offline-first local-primary store.

Brand — SAND WORKS

• **Name:** SAND WORKS

• **Application package:** `com.roshan.sandworks`

• **Display name:** SAND WORKS

• **Tagline/role:** private operator tool (farming/labour trip + money tracking).

Brand assets (locked)

• The brand kit lives in this repo under `assets/`:

• `SAND_WORKS_brand_assets_locked.zip` (archive)

• extracted `sand_works_brand_assets/` tree containing:

• `master/` — the canonical PNG masters (logo + app icon).

• `logo/` — logo at standard sizes.

• `android/` — launcher mipmaps and Play icons.

• **Immutability:** masters are locked. No regeneration, redraw or SVG replacement. Only mechanical resizing/formatting **from a master** is permitted (e.g. producing a launcher density copy).

• The locked masters are considered authoritative; the app icon and splash/About usage derive from them.

Usage guardrails

• The SAND WORKS brand is used only for this product, never for the prior reference application.

• Do not introduce alternative logos or re-styled artwork.

• All screens (especially auth/welcome, splash, About) use the locked logo.

Requirements Index & Traceability — SAND WORKS

Status: **SPECIFIED.** Bidirectional mapping: every requirement → feature → screen → data → backend → security → test → implementation task. See `docs/implementation/TRACEABILITY-MATRIX.md` for the full matrix. This index lists feature → screen source of truth.

Coverage statement

Every feature listed in `docs/product/features.md` and `PRD2.md` appears here mapped to screens and spec docs; every screen in `docs/screens/SCREEN-CATALOG.md` maps to ≥1 requirement. Full requirement→task mapping lives in `docs/implementation/TRACEABILITY-MATRIX.md`. No orphan requirements, no orphan screens.

Brand Asset Verification — SAND WORKS

Status: **EXISTING** (assets present, intact, unmodified). Masters verified 2026-09-08.

Canonical masters

Both masters are **square 1536×1536** (verified programmatically). No 1254×1254 or 1024×1024 "master" files exist in this repository, so **there is no canonical-master ambiguity here**. (The previously discussed 1536 vs 1254/1024 discrepancy belonged to an older working copy and does not apply to this fresh repository.)

Note: `logo/sand_works_logo_1536.png` is byte-identical to `master/sand_works_logo_master.png` (same hash `69b6abdc4923`) — they are the same file content at two paths.

Derived Android launcher mipmaps

Play icons

Logo sizes

`logo/sand_works_logo_256.png` (256), `_384` (384), `_512` (512), `_768` (768), `_1024` (1024), `_1536` (1536). All square.

Guardrails

• Masters are authoritative and locked. Only mechanical resizing from a master is permitted (e.g. launcher density copies already present). No redraw, regeneration, or SVG conversion.

• The launcher mipmaps and play icons are the Android-consumable derivations of the icon master.

Complete Screen Catalogue — SAND WORKS

Status: **SPECIFIED.** This is the exhaustive inventory. Every screen is detailed in the referenced role/shared spec files. No screen is listed purely to inflate the count — each exists because a real workflow needs it.

Conventions: role `O`=OWNER, `D`=DRIVER, `L`=LABOURER, `*`=all. Screen detail per screen is given in `AUTH.md`, `OWNER.md`, `DRIVER.md`, `LABOURER.md`, `SHARED.md` in this folder.

Global / auth

OWNER

DRIVER

LABOURER

No unnecessary screens

Only the above are specified. If a build task proposes an extra screen, it must map to a documented workflow; otherwise it is rejected as scope creep.

Auth & Global Screens — Screen-by-Screen Spec — SAND WORKS

Status: **SPECIFIED.** Each screen lists: purpose, entry/exit, role, app-bar/back, actions, content, states, security, post-action navigation.

Shared screen-behaviour contract (applies to all screens, defined in `docs/spec/SCREEN-STATE.md`): top app bar with deterministic top-left back; content hierarchy; responsive; accessibility; loading/empty/error/retry/success/disabled/permission-denied/network/validation/confirm-destructive/snackbar; state restoration; scroll/keyboard/focus; gestures per GESTURES.md; no fake success.

Splash/Initialization

• Purpose: show locked logo, initialize, restore session.

• Entry: app open. Exit: route by session state.

• Content: logo, subtle progress. States: checking session; if session valid route to role dashboard; else Welcome.

Welcome

• Purpose: entry point to sign in/up; show brand.

• Actions: Sign In, Create Account. Links: (help/about).

• Exit: Sign In, Sign Up.

Sign In

• Fields: email, password. Actions: Sign in; Forgot password; Create account.

• States: idle/submitting/error(invalid)/account-not-approved/account-disabled-suspended/network(retry).

• On success route by role (backend role). Security: TLS; no client role assertion.

Sign Up (DRIVER/LABOURER)

• Fields: name, phone(optional), role request (driver/labourer), email, password, confirm.

• Owner signup: provisioned by operator (no public create-owner).

• On submit (D/L) → Approval Pending. States incl. validation-error, already-registered, network.

• Security: role selection is a request only; grant by owner + backend.

Forgot Password

• Field email → send reset (backend). Confirmation; do not reveal account existence. Network handled.

Email/Account verification (if applicable)

• If verification is used: resend/verify status; not blocking if product chooses password-reset-only (owner decision). PROPOSED default: use secure password-reset; no mandatory email verify.

Approval Pending

• Informational: awaiting owner approval. Action: sign out. No operational data shown.

Account Rejected

• Honest state: rejected by owner. May be re-registered/retried per owner; shows no operational data.

Account Suspended/Blocked / Disabled

• Honest: no access. Reason context if provided. Action: sign out; contact owner.

Session Expired

• Message + sign in. Drop to Sign In.

Network Unavailable (global)

• Banner/overlay + retry. Never fake success. Screens still show cached presentation where appropriate but writes show pending/error honestly.

Maintenance/System Unavailable

• If backend reports maintenance/unavailable: honest system-unavailable state + retry.

Profile (any role) — see SHARED.

Logout confirmation — confirm → sign out → Welcome.

Refer to `docs/architecture/ERROR-STATES.md` for the global state catalogue and `SHARED.md` for Profile/Notifications/Settings/About.

OWNER Screens — Screen-by-Screen Spec — SAND WORKS

Status: **SPECIFIED.** Role: OWNER only unless noted. Every screen follows the shared screen-behaviour contract (SCREEN-STATE.md) and role scoping (roles-access.md). Bottom nav: Home · Trips · People · More.

Home (Owner Dashboard)

• Purpose: operational overview.

• Content: today's trips; today's work; money/accrual summary; active drivers/labourers; recent activity; pending approvals; alerts.

• Actions: quick links; open pending approvals; add tractor.

• States: loading/empty(honest)/error/offline/forbidden. Security: org-scoped reads.

Daily Summary

• Purpose: review a date's accrued-money summary and closure state.

• Shows: total trips, per-person accrued, closure status/time. If a scheduled closure did not run and no server path exists, show honest state + documented fallback (idempotent).

Trip Management / Trip History / Search-Filter Trips

• List all org trips with filters (date range, driver, labourer, tractor, status) + sort (see search-filter-sort.md). Empty/error/offline; paginated.

Add Trip / Edit Trip / Trip Detail (owner may add/edit any)

• Add: date/time, tractor(active), driver, labourers present, rate snapshot, submit → backend number. Edit permitted fields; revision conflict handling. Detail: full read with actions (edit/void). No fabricated numbers.

Tractor List / Add / Edit / Detail

• Per tractors.md. List with active/inactive filter. Add/Edit forms with validation/duplicate handling. Detail with per-tractor totals + history.

Driver List / Detail; Labourer List / Detail

• List org people by role with status filter/search. Detail: profile, status, approval state, totals/accrual (owner-visible), attendance. Actions: disable/suspend/reactivate, (driver detail: see trips).

User Approval / Pending Users

• List pending D/L registrations; Approve/Reject each (with optional note). Audit; notify applicant (type C). Security: owner-only; backend-authoritative.

Temporary Access

• List/create assignments; create: target labourer, start, expiry, reason, scope. Revoke. Per temporary-access.md state machine. Enforced by backend.

Rate Configuration

• Set per-trip rate (■ default 200). Shows current; preview effect (future only). Historical immutable. Owner-only; audit.

Money/Accrual Overview

• Org-wide accrued totals by person/date/period; filter; derived from closure. Owner-only.

Person Accrual Detail

• One person's accrued breakdown by date/trip; own-scope rule for that person's money visible to owner.

Daily Closure

• View closure per date; trigger/re-run idempotently (server-authoritative). Show status. Honest about scheduling backend.

Weekly / Monthly Summary

• Summaries over period: totals, trips, per-person; owner-only; derived from persisted trips.

Leaderboards

• Weekly/monthly top ranks (honest, per leaderboards.md). Owner sees org leaderboard.

Attendance

• Per labourer per date working/absent; owner marks/corrects with reason; audited; conflict handled.

Emergency Warning

• Composer: optional message, recipients, confirmation step → backend-sends strongest-compliant urgent notification. Delivery state, history, retry, duplicate prevention, cancellation where possible, audit. Never claims silent/DND override. (See notifications.md.)

Message Composer / Message History (owner-only broadcast)

• Compose to recipients; send; history with delivery state; audit. Owner-only.

Export Center / Export Configuration

• Choose range, breakdown, format PDF/CSV; generate; download/share; history. Owner-only; server-side where plan supports; honest fallback otherwise. (exports.md.)

Audit / Activity History

• Read-only audit log; filters by user/action/date.

Owner Profile / Application Settings / Security-Account Settings

• Profile (own) incl. photo (plan-dependent). Settings: summary time (19:30 Asia/Kolkata default), notification prefs, org profile. Security/account: change password, session, logout, (2FA PROPOSED/owner decision).

DRIVER Screens — Screen-by-Screen Spec — SAND WORKS

Status: **SPECIFIED.** Role: DRIVER (own scope). Bottom nav: Home · My Trips · My Totals · More.

Driver Dashboard (Home)

• Purpose: today's working view + primary ADD TRIP.

• Shows: today's trips; today's tractor; assigned labourers today; today's summary; recent trips.

• Primary action: prominent **+ ADD TRIP**.

• States: loading/empty(honest)/error/offline/submitting.

Today's Trips

• List today's own trips; tap → Trip Detail; add. Refresh.

Add Trip / Edit Trip / Trip Detail (own)

• Fields: date/time, tractor (active registry), driver=self, labourers present, rate snapshot (auto from current, immutable after record), total; trip number backend-authoritative.

• Driver may edit own permitted trips; revision conflict handling; never silent overwrite. Void only via owner (driver may request/correct errors within scope).

• States incl. conflict, backend-rejected, network.

Trip History

• Own trip history with date filter/sort; paginated; empty/error/offline.

Tractor Selection (within Add Trip)

• Modal/bottom sheet listing active tractors per authorization; pick; validation.

Labourer Selection

• Modal listing active labourers to add as present participants; multi-select; eligibility check.

Daily Total

• Own trips count for the day + work summary (today's total trip count).

Accrued Money (own)

• Own accrued figure + breakdown; derived from closure; own-scope. Wording accrued-only.

Share Today's Trips

• Share today's trip count via Android/WhatsApp share intent (real values; share-sheet fallback). No fabricated numbers.

Notifications / Profile / Settings

• Own notifications (approval, assignment, alerts, daily accrued-money summary). Profile edit own fields; photo plan-dependent. Settings subset.

Temporary Assignment status / Assignment Expired

• If the driver is acting under an owner-authorized capability or a labourer is temporarily assigned, show assignment status. "Assignment Expired" honest state when a temporary grant ends (no silent access). Driver is never an admin.

LABOURER Screens — Screen-by-Screen Spec — SAND WORKS

Status: **SPECIFIED.** Role: LABOURER (own scope, operationally read-only). Bottom nav: Home · My Days · Leaderboard · More.

Labourer Dashboard (Home)

• Purpose: personal metrics, read-only.

• Shows (own, derived from real persisted trips): total trips; accrued money; remaining/accrued outstanding (as defined); working days; absent days; working dates; absent dates; weekly + monthly rank.

• No write actions. States: loading/empty(honest zeros)/error/offline. Wording accrued-only.

Today's Work

• Own participation today (trips/labourer presence); read-only.

Trip History (read-only)

• Own trip participation by date; filters; read-only.

Accrued Money

• Own accrued figure + breakdown by date/period. Derived; accrued-only wording.

Remaining/accrued balance

• The outstanding/remaining accrued amount as defined by the product (the tracked accrued total owed, never " paid"). Label honestly.

Working Days / Absent Days / Date Detail

• Days/absent counts; dates lists; a date detail shows trips/status that day. Read-only.

Weekly Leaderboard / Monthly Top-3 Leaderboard

• Top-3 (honest; fewer participants → fewer ranks) + own rank. Period Asia/Kolkata. No fabricated ranks.

Notifications / Profile / Settings / Account-Access Status

• Own notifications; own profile edit within bounds; settings subset; account/access status (active/approval/temporary-assignment). Read-only operational.

Shared Screens — Screen-by-Screen Spec — SAND WORKS

Status: **SPECIFIED.** Screens reachable by more than one role; re-scope by role.

Trip Detail (shared owner/driver)

• Shows date/time, tractor, driver, participating labourers, rate snapshot, total, trip number.

• Actions by role: owner (edit/void any), driver (edit own permitted). Conflict handled.

• Security: role + org + ownership.

Notifications / Notification Detail

• Centre listing own notifications (types A–F: A daily accrued-money summary, B operational alert, C approval outcome, D trip assignment, E temporary-assignment expiry, F operational).

• Open → deep link target (re-validated). Mark read/unread; badge.

• Wording: daily accrued-money summary, never "payment". No cross-user money broadcast.

Settings (role subset)

• Owner full settings (summary time 19:30 Asia/Kolkata default, notification prefs, org profile). D/L see only their own allowed settings (e.g. notification prefs).

Help / About

• SAND WORKS brand, version, private one-owner nature, contact owner. Uses locked logo.

Profile / Edit Profile (any role)

• Own name, role, phone, status, photo (plan-dependent), own stats. Edit allowed own fields within owner-controlled bounds. Security: own record.

Change Password

• Backend-secure change; sign-in again after if required. Validation.

Logout confirmation

• Confirm → sign out → Welcome. Sessions cleared; no data fabrication.

Global overlays

• Network Unavailable, Maintenance, Session Expired, Account Blocked, Assignment Expired — see `docs/architecture/ERROR-STATES.md`.

Navigation Architecture — SAND WORKS

Status: **SPECIFIED.**

1. Root navigation & auth

• Root: an Auth gate. Not signed in → Auth flow. Signed in → role routing.

• Deep-link / shared content: re-validates auth + role + org + ownership + existence before showing content; otherwise honest Forbidden/NotFound.

2. Authentication navigation

`Welcome → Sign In | Sign Up | Forgot`; Sign Up (D/L) → `Approval Pending`; valid session → role dashboard. Session expired/blocked → Session Blocked → Sign In.

3. Role routing

• OWNER → owner bottom-nav shell.

• DRIVER → driver bottom-nav shell.

• LABOURER → labourer bottom-nav shell.

4. Bottom navigation (4–5 primary destinations max per role)

• **OWNER (4):** Home · Trips · People · More. (Sub-functions — money, reports, audit, settings, tractors, approvals — under More or contextual detail screens; not a 9-item bar.)

• **DRIVER (4):** Home · My Trips · My Totals · More.

• **LABOURER (4):** Home · My Days · Leaderboard · More.

Selected/unselected states per Material 3; badge for notifications/pending approvals where appropriate.

5. Hamburger / drawer

• Hamburger appears only where a drawer is the right pattern (secondary/tertiary actions). For each role the drawer contains only that role's items. OWNER-only functions never appear in D/L drawers.

• Contents (role-scoped): profile, notifications, settings, help, export (owner), security/account, logout.

• Behaviour: tap item → navigate; outside-tap closes; back closes drawer; a11y labelled; no owner-only items in D/L.

6. Detail & modal navigation

• Detail push (e.g. Trip Detail from list). Modals/bottom sheets for quick pickers (tractor select, labourer select). Full forms are screens, not cramped sheets.

7. Nested flows & back stack

• Add/Edit Trip is a flow with back to originating list; after success navigate to Trip Detail or return with result.

• Temporary-assignment flow, export flow, emergency-warning flow are each nested with defined back.

8. Deep links (if required)

• Notification → target screen after re-validation. Reset link → Forgot/verify.

9. Routing on events

• Session expiry → Session Blocked (drop to sign-in). Unauthorized → Forbidden. Temporary-access expiry → Assignment Expired / reduced to labourer.

10. Back / top-left arrow deterministic behaviour

• Top-left back arrow and system Back behave identically to pop the current destination to its logical parent.

• Inside a form with unsaved changes → warn/confirm before leaving (don't lose data silently; don't fake-save).

• Modal open → back dismisses modal first.

• Keyboard visible → back first dismisses keyboard.

• Operation in progress → block navigation-with-context (show in-progress; prevent duplicate submit) but allow cancel where safe.

• Session expires mid-flow → force to Session Blocked.

Design System — SAND WORKS

Status: **SPECIFIED.** Professional, production-grade. Brand direction: industrial, premium, modern, clean, powerful, trustworthy, operational. Uses the approved PNG brand assets only (no regeneration/redraw/SVG).

1. Colour tokens (Material 3 colour roles; exact hex in a theme tokens doc during implementation)

Recommendation (PROPOSED — owner approves exact palette in an ADR):

• **Primary (industrial amber/gold)** signalling energy/earnings: e.g. amber `#F5A623`-family.

• **Secondary/Neutral (graphite/stone)** for surfaces: near-black `#1E1E1E`, concrete greys.

• **Semantic:** success green (accrued/working), error red (warning/absent/error), warning amber.

Final tokens: `primary`, `onPrimary`, `primaryContainer`, `onPrimaryContainer`, `secondary`, `surface`, `surfaceVariant`, `background`, `error`, `outline`, and dark-theme variants. Contrast meets AA at minimum (see ACCESSIBILITY.md).

2. Typography

• Typeface: system default (Roboto on Android) for reliability; scale uses Material 3 type roles (display/headline/title/body/label). Numeric figures use tabular where aligned (money/dates). Money formatted from paise at UI.

3. Spacing & grid

• 4dp base spacing scale (4/8/12/16/24/32). 8dp grid for layout. Standard content max width; responsive.

4. Elevation / borders / corners / surfaces

• Material 3 elevation tonal or shadow tokens; cards on `surfaceContainerLow`; subtle borders; radius scale 8/12/16/28. Avoid flat generic template look.

5. Components

Buttons (filled/tonal/outlined/text + destructive), cards, forms & fields (outlined, error, char counters), dialogs (confirm, destructive), bottom sheets, navigation (bottom bar ≤5, top app bar, drawer), badges, chips (filter), tables/lists, avatars (with profile photo fallback initial), icons (Material icons, consistent stroke), charts (simple accrual/day bars — only where useful), leaderboard components (rank rows), notification components, loading indicators (linear/spinner/skeleton), empty-state and error-state visuals (clear illustration + action, no fabricated images).

6. Dark & light theme

Both supported; tokens adapt; content contrast maintained in both. (Dark/light default = PROPOSED, owner may choose; implement both per DESIGN tokens.)

7. Motion & haptics

• Motion: short, purposeful (e.g. 150–300 ms) for navigation, feedback; respect reduced-motion (see ACCESSIBILITY.md).

• Haptics: light on confirm, warning/error on failure, only where meaningful; alternatives for reduced-motion/haptics-off.

8. Responsive layouts

• Phones primary. Large screens/tablets: use wider content, two-pane where sensible (e.g. owner trips list+detail). Touch targets ≥48dp. Text scaling supported.

9. Accessibility & content

• Semantics, content descriptions, contrast, colour-independent status. Details in ACCESSIBILITY.md.

10. Brand application

• Splash/About/auth welcome use the locked logo from `assets/sand_works_brand_assets`. No placeholder logos anywhere.

GESTURES — SAND WORKS

Status: **SPECIFIED.** Only meaningful gestures; no gimmicks. For each: screen, target, action, visual feedback, haptic, accessibility alternative, accidental-trigger protection, confirmation for destructive actions.

1. Tap

• Primary selection/activation (buttons, list items, tabs, bottom-nav, chips). Visual: ripple/pressed. A11y: standard button/list semantics. Accidental: min 48dp target.

2. Long press

• Context actions on list items where useful (e.g. a trip row → quick actions menu). Visual: selection/context. A11y: an explicit "More" affordance is also present (never rely on long-press only). Confirm destructive actions.

3. Swipe / horizontal actions

• Where used (e.g. notifications → dismiss; a list row → primary quick action). Visual: item slides + action revealed. A11y: buttons also exposed. Provide undo/snackbar for destructive; confirmation where appropriate.

4. Pull-to-refresh

• On dashboards, trip lists, notifications, leaderboards, reports (refresh from backend). Visual: standard pull indicator. A11y: refresh action also exposed. Respects online state (refresh only makes backend calls; offline shows error/retry, never fake).

5. Scroll

• Lists/history scroll; pull-to-refresh disabled while loading. Loading/empty/error handled.

6. Drag / dismiss

• Bottom sheet drag to dismiss; dialogs only via explicit action or system back (not accidental drag). Dismiss of in-progress forms → confirm (unsaved changes).

7. Back gesture / system back

• Deterministic per NAVIGATION.md §10. Edge-to-edge back gesture supported; no ambiguous leaves.

8. Keyboard interaction

• Forms: next/done; focus order logical; fields scroll into view; submit on IME action where single-field; block double submit.

9. Pinch / zoom

• Only if genuinely needed (e.g. profile image crop preview uses crop handles/drag, not necessarily pinch; export/report images not zoomed). Avoid unneeded pinch.

10. Image crop gestures

• Profile photo: crop overlay with drag handles + drag image; confirm/cancel; a11y buttons.

11. List interactions

• Tap → open detail; swipe quick-action; long-press context where used. Sort/filter via explicit controls (not gestures).

Global rules

• Every gesture has a non-gesture alternative.

• Haptics: light on confirm/complete, error on failure, none where reduced-haptics.

• Destructive actions require confirmation; visual + (where set) haptic feedback on success/failure.

• Accidental-trigger protection: swipe targets have thresholds; undo offered for non-destructive dismisss.

Screen-State & Behaviour Contract — SAND WORKS

Status: **SPECIFIED.** Applies to every screen; referenced by all screen specs.

State machine (sealed UI state per screen)

Each screen exposes a truthful state: `Loading` → `Content | Empty | Error | Offline` plus transient `Submitting`, and role/account outcomes `Forbidden`, `Unauthorized/SessionExpired`, `AssignmentExpired`. No fake success, no fake loading completion.

• **Loading:** initial load; skeleton where appropriate; button progress on actions; prevent duplicate submission.

• **Empty:** clear, honest empty states (no trips/users/approvals/tractors/labourers/leaderboard/accrual/notifications/search/export). Actionable reset where filters apply.

• **Error:** typed — auth failure, permission denied, network, backend unavailable, timeout, rate limited, validation, conflict (concurrent edit), duplicate request, expired session, expired temporary access, feature-unavailable (plan), server error, export failure, notification failure, image upload failure. Message + retry where applicable.

• **Offline/network degradation:** online-first app shows connection indicator; retry; exponential backoff where appropriate; stale cached display only clearly marked; **never show "saved successfully" unless the backend accepted the authoritative write**; failed write handled honestly; duplicate prevention; recovery after reconnect.

Behaviour contract per screen

• Purpose, entry/exit points, role visibility.

• Top app bar: deterministic top-left back (NAVIGATION.md §10); title; actions.

• Hamburger only per NAVIGATION.md (role-scoped; owner-only items never in D/L).

• Bottom navigation ≤5 role destinations; selected/unselected states; badge.

• Content hierarchy, cards/lists/forms/buttons/icons/typography/spacing/colours/surfaces per DESIGN-SYSTEM.md; responsive.

• Accessibility per ACCESSIBILITY.md; gestures per GESTURES.md.

• Validation with inline errors; confirmation dialogs for destructive actions; snackbar for ephemeral results.

• Navigation after success (documented target) and after failure (stay + error).

• State restoration on process death/rotation/back.

• Scroll/keyboard/focus behaviour; animation/haptics within DESIGN-SYSTEM rules.

Session/role events

• Session expiry → Session Expired. Unauthorized → Forbidden. Temp-access expiry → Assignment Expired. Account disabled/suspended → Blocked.

No placeholder rule

Every visible control works; every state is real. No inert/decorative controls (AGENT.md).

UX-FLOWS — End-to-End Workflows — SAND WORKS

Status: **SPECIFIED.** Each flow documents the main path and every conditional/failure branch. Backend-authoritative where noted.

Flow 1 — New user → approval → dashboard

New DRIVER/LABOURER: Welcome → Sign Up (role request) → Approval Pending → (owner approves) → Sign In → role dashboard. Branches: rejected → Account Rejected (no access); disabled/suspended → Blocked; session expires → re-sign-in.

Flow 2 — Owner approves a user

Owner → Approvals → review → Approve/Reject. Success: user active, audit, notify (C). Branch: reject → user sees rejection.

Flow 3 — Driver creates a trip

Driver → +ADD TRIP → select date/time, tractor (active), labourers present → backend validates (role, active tractor/labourer, approval), snapshots rate, assigns number → saved → accrual queued → (closure) daily accrued-money summary notification. Branches: tractor inactive → choose active; labourer ineligible → error; duplicate → rejected (idempotent); offline → honest error/pending, retry; conflict → surface.

Flow 4 — Driver absent → temporary labour assignment → expiry

Owner (or authorized driver) creates assignment (labourer, start, expiry, reason, scope) → backend enforces → labourer performs driver duties during window → expiry → access removed (Assignment Expired) → role back to LABOURER. Branches: overlap → defined resolution; revoked early by owner; login after expiry → denied.

Flow 5 — Owner configures rate

Owner → Rate config → set new ■ → snapshot future → historical trips unchanged. Branch: validate integer/paise.

Flow 6 — Daily operation → closure

Trips recorded through day → scheduled closure (19:30 Asia/Kolkata) or documented fallback → snapshot trips → compute per-person accrued (integer, idempotent) → daily accrued-money summary → notify eligible. Branch: closure re-run → idempotent, no double count; partial data → honest.

Flow 7 — Owner emergency warning

Owner → Emergency Warning → compose optional message → recipients → confirm → backend authorize → strongest-compliant urgent notification (sound/vibration/heads-up where OS permits; never silent/DND override) → delivery state + audit. Branches: delivery failure → retry; duplicate prevention; cancellation where possible.

Flow 8 — Labourer reads dashboard/leaderboard

Labourer → dashboard (own metrics) → working/absent → weekly leaderboard (top-3 honest) → monthly top-3. Branches: <3 qualify → fewer ranks; none → empty; period reset Asia/Kolkata.

Flow 9 — Owner exports

Owner → Export → range + breakdown + format → authorize → generate (server-side where plan supports) → download/share → history. Branch: empty range; failure; non-owner denied.

Flow 10 — Profile photo

User → Profile → upload/replace/delete → validate → (plan) storage → success. Branch: plan lacks Storage → feature marked unavailable (not faked); failure → retry.

Flow 11 — Notification deep link

Notification → open → re-validate auth+role+org+ownership+existence → target screen or honest Forbidden/NotFound.

Flow 12 — Account recovery

Forgot → reset via backend → sign in. Branch: disabled → Blocked.

WIREFRAMES — SAND WORKS (specification wireframes)

Status: **SPECIFIED — these are specification wireframes, NOT implemented UI.** ASCII layout to communicate hierarchy. Owner approval of final visual mockups is separate (no mockup is claimed to exist here).

Auth — Sign In

```
[ LOGO — SAND WORKS ]
Email           [_________________]
Password        [_________________]
[          Sign In (filled)     ]
Forgot password?      Create account
(network/error/approval-pending status area above button)
```

Owner Dashboard (Home)

```
[■] SAND WORKS              [bell ■] [■]
Trips today: 12   Work: 3   Accrued ■[amount]
Active drivers: 2  Active labourers: 5  Pending approvals: 1
Recent activity        [1 pending approval →]
  • 12:40 Trip #124 ...
Daily summary card        [open]
[B Home] [Trips] [People] [More]
```

Driver Dashboard

```
[■] Good day, Mansingh
Today: 4 trips    Tractor: Sonalika
Assigned labourers today: 3
[        + ADD TRIP (primary)        ]
Recent trips list ...
[B Home] [My Trips] [My Totals] [More]
```

Labourer Dashboard (read-only)

```
[■] Your summary
Total trips: 8    Accrued: ■640
Working days: 3   Absent days: 1
Dates: [1 Mar ✓] [2 Mar ✓] ...
Weekly rank: #2   Monthly: top-3
[B Home] [My Days] [Leaderboard] [More]
```

Add Trip (owner/driver)

```
[←] Add Trip
Date [01/03]  Time [12:30]
Tractor ■ [Sonalika]   Driver [Mansingh Rana]
Labourers present: [+ add]  ■ A ■ B
Rate snapshot: ■200 (immutable)
Total: ■400  Trip # (auto on save)
[ Cancel ]           [ Save Trip ]
```

Trip Detail

```
[←] Trip #124
Date/Time  Tractor  Driver  Labourers
Rate snapshot ■200  Total ■400
Status: active  [Edit] [Void (owner)]
```

User Approval

```
[←] Pending Approvals
Name Role  Requested   [Approve] [Reject]
```

...

Tractor Management

```
[←] Tractors    [+ Add]
Sonalika        active  [Edit] [Deactivate]
John Deere      active  [Edit] [Deactivate]
```

Money / Accrual

```
[←] Accrual Overview (owner)
Date range [01/03–31/03]
Driver    trips   accrued
Labourer  trips   accrued
[Export PDF] [Export CSV]
```

Leaderboard (monthly top-3)

```
[←] Monthly Leaderboard (Mar)
#1  A — 24 trips
#2  B — 20 trips
(no fabricated #3 if fewer)
```

Emergency Warning (owner)

```
[←] Send Warning
Message (optional) [_____________]
Recipients: ■ all drivers ■ all labourers
[ Cancel ]  [ Confirm & Send ]
Warning is urgent; Android may not override Silent/DND.
```

Notifications

```
[←] Notifications
■ Daily accrued-money summary — today ... [unread]
■ Operational warning from owner ...
[read/unread filter]
```

Profile

```
[←] Profile
[photo] Name  Role: DRIVER
Phone  Status: active
[Edit] [Change password] [Logout]
```

Settings (owner)

```
[←] Settings
Daily summary time [19:30]
Notifications [on]
Org profile ...
```

Export Center

```
[←] Export
Range [date]  Breakdown [per driver ■]
Format [PDF|CSV]  [Generate]
History ...
```

Representative empty/error/loading

```
Loading: skeleton rows
Empty (trips): "No trips yet. [Add trip]"
Error: "Couldn't load. [Retry]"
Offline: banner "Offline — showing latest cached data (not live). [Retry]"
```

Money Engine — SAND WORKS (authoritative spec)

Status: **SPECIFIED.**

1. Money representation

• All authoritative money is **integer paise**. ■200 = `20000` paise.

• Never use floating point for authoritative monetary calculation or storage.

• Display formatting (■) happens only at the UI boundary from integer paise.

2. Rate & immutable snapshot

• Default per-trip rate: ■**200** (owner-configurable).

• The OWNER may change the rate at any time.

• Every trip records an **immutable** `rateSnapshot` at record time. Changing today's rate **never rewrites historical trips**. Historical trips keep their snapshot forever.

3. Trip money fields

For each trip:

• tractor selected; driver identified; participating labourers identified (present/eligible); applicable rate captured as `rateSnapshot`; total = pool; trip recorded; authoritative trip number (backend); money distribution computed per the approved rule.

4. Distribution rule (owner-configurable — exact allowed set)

• **Default:** equal split among the driver and the eligible labourers marked present/participating on that trip.

• The OWNER may configure the distribution rule. Permitted configuration options are limited to:

1. **Equal** — pool divided equally among driver + each eligible present labourer (default).
2. **Driver + labour share** — distinct share for driver vs each labourer (fixed ratio, owner-set).
3. **Custom %** — owner-set percentages summing to 100%.
4. **Fixed allocation** — owner-set fixed paise amounts.

• No other wage model may be invented.

• Distribution is computed by the backend for the daily closure, not trusted from the client.

5. Rounding in paise (deterministic)

• Pool is integer paise. Division that does not divide evenly leaves a remainder in paise.

• Deterministic remainder rule (documented and unit-tested): distribute the remainder one paise at a time in a fixed, stable order (e.g. lowest-priority recipients first, or a documented fixed participant order) so that the **sum always reconciles exactly to the pool**. The exact order is a documented constant; tests assert `Σ shares == pool`.

6. Labourer-count cases

• **Zero labourers present:** pool applies per rule (equal → the driver receives the full pool; other rules apply their driver share handling per configuration).

• **One labourer present:** equal → driver and the single labourer split; other rules per config.

• **Multiple labourers present:** equal → split evenly among driver + each present labourer; other rules per config.

7. Duplicate submission & in-flight guard

• A trip submission is idempotent: client generates an idempotency key; backend rejects a duplicate (same key/operation) rather than creating two trips.

• Client disables submit while in flight to prevent accidental double-tap; the backend is the authority.

8. Edits

• Permitted trip edits (driver-own within scope, owner-any) follow optimistic concurrency: each mutable trip carries a `revision`; on conflict the user is shown the situation and chooses; never silent last-writer-wins.

• Edits never change an already-snapshotted `rateSnapshot` unless the edit is a correction of an error before closure, which is itself owner/audit governed.

9. Deletion / voiding

• Trips are not hard-deleted from history. A trip may be **voided** (soft) by the owner with a reason; voided trips are excluded from accrual/closure and marked in audit. Never double-count a void then re-close without idempotency.

10. Historical-rate behaviour

• Past closures, leaderboards and person totals are computed from immutable snapshots and are not recomputed when the current rate changes. Only new/future trips use the new rate.

11. Concurrency & retry / idempotency

• Mutable documents carry `revision`; conflicts surfaced to the user.

• Server-authoritative operations (closure, accrual, numbering, leaderboard) carry idempotency keys and are transactional; retries never double-apply.

12. Daily accrued-money summary / closure

• Recommended default schedule: **19:30 Asia/Kolkata**, owner-configurable within a documented window (e.g. 18:00–22:00).

• Closure is **server-side scheduled** where the plan supports Cloud Functions; otherwise an honest fallback is documented (never a fake local schedule claiming server authority).

• Idempotency boundary: **(organisation, date)**. Running closure for the same date twice must not double-count.

• Exactly-once semantics; audit records each closure.

• Wording is **"daily accrued-money summary"** / "Today's earnings added" — **never "payment completed"**.

13. Integrity rules

• Money math deterministic, integer, unit-tested (see `docs/quality/TESTING.md`).

• Backend authoritative for money, numbering, closure, leaderboards.

• Idempotency protects money writes from retry duplication.

• Every money-related server operation is audited.

Roles & Access Matrix — SAND WORKS (authoritative)

Status: **SPECIFIED.** Backend-enforced. UI hiding is never a security control.

Roles: **OWNER** (Ramesh Sahu, exactly one), **DRIVER** (e.g. Mansingh Rana, plus approved drivers), **LABOURER**. No ADMIN. Mansingh Rana is a DRIVER.

Legend: R=read, C=create, U=update, D=delete/void, A=approve, X=execute/use.

• O = OWNER only; D = DRIVER (own scope); L = LABOURER (own scope, read-only).

Notes

• All reads/writes are org-scoped and (for D/L) own-scoped. Cross-org and cross-user private-money access is denied by rules.

• Owner capabilities are bounded by the product; every capability has validation, authorization, error handling and audit (see SECURITY/API/docs).

• No role may write another user's private financial data.

• DRIVER never becomes admin; LABOURER writes nothing operational.

Tractor Model — SAND WORKS

Status: **SPECIFIED.**

1. Registry model

• Tractor registry is **OWNER-controlled**. The system supports approximately 4–5 operating tractors and multiple driver accounts.

• Initial records: **Sonalika** and **John Deere**, entered by the owner as registry records. These are **not** hard-coded as the permanent complete set; the owner may add/edit/deactivate as needed.

2. Fields

3. Operations

• **Add:** owner only; validation — unique name within org (case-insensitive), optional identifier; duplicate handling → reject with clear error; active by default.

• **Edit:** owner only; revision conflict handled (surface, don't overwrite); audit.

• **Deactivate (soft):** owner; tractor no longer offered for new trips; existing trips keep their tractor reference/history.

• **Restore/reactivate:** owner; tractor available again for new trips.

4. Relationships

• Trip → tractor (reference). Per-tractor totals and trip history derive from trips referencing the tractor (even if the tractor is later deactivated).

• Driver association "where applicable": a trip records which driver operated which tractor; there is no enforced one-driver-per-tractor binding, but a tractor may be associated with trips by any approved driver.

5. Validation & duplicate handling

• Name required, trimmed, ≤ some max length; uniqueness within org.

• Duplicate add → validation error, no silent overwrite.

• Deactivated tractor referenced by an in-flight draft → warn user to choose an active tractor at submit.

6. Owner SOP pointers

See `docs/spec/SOP.md` (add tractor, deactivate tractor).

Temporary Labourer Access — SAND WORKS (state machine + security)

Status: **SPECIFIED.**

1. Purpose

If the regular driver is absent, the OWNER (or an authorized DRIVER, within documented limits) may temporarily assign a specific LABOURER the access needed to perform the driver's operational duties. This must **not** permanently change the person's role and must **not** create an admin.

2. Assignment entity

3. State machine

`draft → active (start reached) → expired | revoked`

• A proposed assignment is valid only between `start` and `expiry`.

• **Active:** the labourer may perform the assigned driver duties.

• **Expired:** (backend-enforced by time) the labourer no longer has the granted duties.

• **Revoked:** the OWNER may revoke early; status → revoked.

• Overlap resolution: an active assignment is superseded/denied if a newer conflicting one is granted; the effective grant is the latest valid active assignment. Documented, backend-enforced.

4. Backend enforcement

• Authorization is enforced by backend Security Rules / Cloud Functions using server time, **not** the client clock.

• After expiry: a user logging in receives no granted duties (Assignment Expired state), role returns to LABOURER.

• No privilege escalation: the grant is limited to the documented driver-duties scope and time window.

5. Audit

• Every create/revoke/expiry is audited (who, target, window, reason).

6. Notification

• Optional type-E (expiry) notification to the assignee; type-D assignment notification when granted.

7. Failure/edge behaviour

• Assigning to a non-labourer → rejected.

• Assignment without expiry → rejected.

• Overlapping active assignment → defined resolution (documented above); user informed.

• Access after expiry → truthful denied state, never silent access.

Profile Photo — SAND WORKS

Status: **SPECIFIED (plan-dependent).** Every OWNER, DRIVER and LABOURER has a profile screen supporting profile info and, where the chosen Firebase architecture supports it, a profile picture.

1. Plan dependency (honest)

Profile-photo storage uses **Firebase Storage**. Storage availability depends on the Firebase plan. If Storage requires a paid (Blaze) plan, this feature is marked a **plan/environment dependency** and is not represented as available on a free plan. The rest of the profile (name/role/phone/status) is available regardless.

2. User flows

• **Upload:** choose image (gallery/camera per Android), preview, then confirm.

• **Replacement:** overwrite the user's own current photo.

• **Deletion:** remove the user's photo.

• All within own scope (each user edits their own photo); owner may manage their own.

3. Validation & processing

• Image validation: format (jpg/png/webp), MIME, dimensions (min/max documented), file size limit (documented, e.g. ≤ 5 MB original).

• Server/enforcement: Storage rules validate size/MIME/dimensions and that the path is the caller's own.

• Compression and crop before upload (client-side) to a documented max dimension.

4. Upload UX

• Loading indicator; upload progress; success; failure with retry; cancellation.

• Never show "photo saved" unless storage accepted it.

• Network failure → honest error + retry.

5. Authorization & privacy

• Read: the user's own photo (and, where shown in owner/leaderboard contexts, only approved thumbnails as permitted). No arbitrary cross-user private image disclosure.

• Write/delete: the user's own record only.

• Storage rules enforce owner/self.

6. Storage lifecycle

• On replacement/deletion, the old object is removed (or GC'd) per documented policy.

• Photo referenced by uid path: `profiles/{orgId}/{uid}`.

7. Failure handling

• Oversized / wrong type / invalid dims → validation error before upload.

• Upload failure → error state, no silent success, retry available.

Search, Filter & Sort — SAND WORKS

Status: **SPECIFIED.** Search/filter/sort is defined wherever it is useful (users, tractors, trips, dates, driver, labourer, status, working/absent, leaderboard period, money/accrual history). Not every screen needs every control — each is applied only where it adds value.

1. General behaviour (applies to all)

• **Input:** text fields trimmed; search typically begins after debounce (~300 ms) or explicit submit for large queries; case-insensitive for text match.

• **Filter controls:** chips / dropdown / segmented per screen; combined filters are AND-ed.

• **Sort:** pick from a documented set per screen; default sort is documented (usually newest first by date/time).

• **Empty result state:** clear, actionable "no matches — clear filters" (reset) with no fabrication.

• **Reset:** one-tap clear of all filters/sort back to default.

• **Sort stability:** tie-break by a deterministic secondary key (e.g. id or timestamp) so ordering is stable across refreshes.

• **Pagination:** paginated Firestore queries where result sets can grow (trips, history); infinite scroll or load-more with loading state; no silent truncation.

• **Loading/error:** skeleton during load; error with retry; network-offline message.

• **Permissions:** only data the role may read is ever returned/scoped by backend; filters never expose data the user may not see.

• **Backend query implications:** filters map to Firestore where()/orderBy with required composite indexes documented in DATABASE.md; range filters (date) bounded; text search implemented as documented (Firestore limitations respected — no fake full-text).

2. Per-screen search/filter/sort

3. Definitions

• Text match: contains, case-insensitive, on name fields.

• Date filter: uses Asia/Kolkata day boundaries.

• Debounce: 300 ms default for live text search.

Attendance — SAND WORKS

Status: **SPECIFIED.**

1. Concept

Per labourer per date, an attendance record records whether they **worked** or were **absent**. Attendance drives the labourer's working/absent days, working/absent dates, and eligibility for accrual on trips that day.

2. Model

3. Owner operations

• Mark a day working or absent for a labourer.

• **Correct** an existing record with a required reason → audited; no silent overwrite.

• Concurrent-change handling: on revision conflict, surface and let owner choose (never silently overwrite).

4. Derivation vs manual

• A labourer present on ≥1 trip that day is considered working (derived). Owner may still correct.

• Manual absent/working records exist where no trip is present (e.g. driver reported, or labourer marked absent).

5. Counts surfaced (labourer dashboard)

• working days, absent days, working dates, absent dates — derived from attendance/trips within a selected period.

6. Integrity

• Only OWNER marks/corrects. Labourer read-only. DRIVER does not change another's attendance.

• Corrections audited; idempotent writes; no double-count of a day.

Leaderboards — SAND WORKS

Status: **SPECIFIED.**

1. Scope & periods

• **Weekly leaderboard:** resets each week. Period boundary = Asia/Kolkata week (documented start day, e.g. Monday).

• **Monthly leaderboard:** calendar month in Asia/Kolkata, **top 3 only**.

2. Honest ranks — never fabricate

• If only **one** user qualifies → show 1st place only.

• If **two** qualify → show 1st and 2nd only.

• Never display a fake 3rd place when fewer than three qualify.

• If none qualify → empty state ("no data yet"), not a fabricated table.

3. Ranking metric

• Primary metric: number of qualifying trips (or accrued amount) in the period — documented constant. Use **trip count** as the leaderboard metric (matches "share today's trips" and trip-based labour tracking). Ties resolved by a deterministic secondary key.

4. Tie behaviour (deterministic)

• Equal primary metric → tie-break by deterministic secondary key (documented: e.g. earlier first-qualifying-trip timestamp, then user id). Same result every computation.

5. Minimum-data behaviour

• A user qualifies if they meet the documented minimum (e.g. ≥1 eligible trip in the period). Periods with no eligible trips produce an empty/zero state.

6. Data source & integrity

• Derived **by the backend** from real persisted trips, at weekly/monthly boundaries.

• Deterministic and idempotent; recomputable; never client-invented.

• Only real, eligible participants appear; no fabricated ranks.

7. Privacy

• Leaderboard shows rank + count/accrued for the top participants as permitted by owner viewing rules; a labourer sees top-3 and their own rank; the owner sees organisation leaderboard.

8. Timezone/dates

• All period boundaries and resets use **Asia/Kolkata**, consistent with daily closure. Period labels shown in local calendar terms.

Exports — SAND WORKS

Status: **SPECIFIED.** Owner-only.

1. Scope

• Formats: **PDF** (primary human-readable) and **CSV** (raw data).

• Owner-only. Not available to DRIVER or LABOURER.

• Date range + breakdown options: overall, per tractor, per driver, per labourer, per date.

• Content: money totals (accrued), trip/work counts, working/absent days.

2. Access & authorization

• Authorization enforced by backend/Cloud Function — owner role checked; not UI-only.

• Export generation is server-side where the plan supports Cloud Functions/Storage; otherwise a documented honest fallback (owner-scoped local generation) with clear limits. Never fake server-side generation.

3. Flow

1. Owner opens Export Center, chooses range + breakdown + format.

2. Backend authorizes and generates the file.

3. File available for download/share per platform.

4. Export recorded in history with audit.

4. States

• loading / generating / success-with-download / failure / empty (no data in range) / retry / permission denied (non-owner). No fabricated totals.

5. Integrity

• Money figures are accrued only, integer-derived; never "paid".

• No cross-org data; no PII beyond owner scope.

• Concurrency: one export at a time per user or queued with clear progress.

SOP — Standard Operating Procedures (OWNER), SAND WORKS

Status: **SPECIFIED.** Operational procedures for the owner and primary staff.

Identity

Owner: Ramesh Sahu. Primary driver: Mansingh Rana (DRIVER). If a procedure says "owner," it means the OWNER role only.

Procedures

1. Approve a user

1. Open Approvals/Pending Users.
2. Review the applicant's details (name, role request: driver/labourer, contact).
3. Approve (or Reject with optional note).
4. Result recorded to audit; applicant notified (type C). On approval the user may sign in to their role dashboard.

2. Reject / block a user

1. Open the user's record in Manage Users.
2. Reject (pending) or set disabled/suspended (active).
3. Reason recorded; audit; no operational access thereafter.

3. Add a tractor

1. Owner → Tractors → Add.
2. Enter name (e.g. Sonalika, John Deere, model), optional identifier.
3. Save. New tractor appears for new trips. Duplicate name rejected.

4. Deactivate / reactivate a tractor

1. Owner → Tractors → the tractor → Deactivate (or Reactivate).
2. Deactivated tractors are not offered on new trips; history is retained.

5. Add / edit a trip (owner or authorized driver)

1. Open Trip screen → Add Trip (or edit an own/permitted trip).
2. Choose date/time, tractor (active), driver (self or recorded), participating labourers, verify rate snapshot.
3. Submit → backend validates, assigns trip number, snapshots rate.
4. On success the trip is recorded; if a duplicate or conflict occurs, follow the on-screen guidance (never resubmit blindly).

6. Correct an erroneous trip

1. Owner (or driver for own error) opens the trip.
2. Correct the permitted fields within scope; if before closure, follow owner/audit governance; never alter an immutable snapshot unless it is an approved correction.
3. Save; conflict handling if the trip changed concurrently.

7. Configure the rate

1. Owner → Rates/Rate Configuration.
2. Set new per-trip rate (default ■200).

3. Confirm. New trips use the new rate; historical trips keep their snapshot.

8. Configure distribution rule

1. Owner → Money-rule settings.

2. Choose equal (default) / driver+labour share / custom % / fixed. Set parameters.

3. Confirm; applies to future trips/closures.

9. Inspect daily accrual / summaries

1. Owner → Money/Accrual overview and Daily Summary.

2. Review accrued totals; verify a closure ran (19:30 Asia/Kolkata default). If a scheduled closure did not run and no server path is available, use the documented honest fallback (idempotent; never double-count).

10. Inspect weekly / monthly leaderboards

1. Owner → Leaderboards; select week/month.

2. Review top ranks (real data). No fabricated ranks.

11. Send an emergency warning

1. Owner → Emergency Warning.

2. Compose optional message; select recipients; confirm (confirmation step).

3. Backend authorizes and sends the strongest Android-compliant urgent notification (high-importance channel; sound/vibration/heads-up where the OS permits). Never expect full-volume override of Silent/DND.

4. Delivery state + history + audit; retry if needed; duplicate prevention.

12. Send a message (broadcast, owner only)

1. Owner → Message Composer; pick recipients; send. Recorded to history/audit.

13. Export data

1. Owner → Export Center; choose range, breakdown, format (PDF/CSV).

2. Generate; download/share. Recorded.

14. Manage profile & recover account

1. Owner → Profile: edit own fields; change password (backend); recover account via password reset if locked out.

2. Lost phone → reinstall, sign in with same account; data is in the backend (online-first). No local-only data loss path is the authority.

15. Handle network failure

1. Screens show a truthful offline/error state with retry. No "saved successfully" unless the backend accepted the write. Wait for connectivity and retry.

16. Handle notification failure

1. Check device permission (notifications enabled); verify FCM configured; retry/send again from history; if real FCM is not provisioned (plan), use the documented fallback and do not claim delivery.

17. Handle a temporary driver assignment (driver absent)

1. Owner → Temporary Access → create assignment for a specific labourer (start, expiry, reason, scope).

2. Backend enforces expiry. If regular driver (e.g. Mansingh Rana) returns, revoke or let it expire.

18. Handle an absent driver (operational)

1. Determine whether a temporary assignment is needed (procedure 17) or another approved driver takes the trip.

2. Ensure labourer performing driver duties does so only under a valid assignment.

19. Review audit history

1. Owner → Audit/Activity → filter by user/action/date. Read-only.

20. Prepare a private APK release

1. Confirm owner-provided Firebase project + plan; deploy Firestore, Auth, App Check, FCM, Security Rules, indexes, Cloud Functions (as applicable).
2. Build with the dedicated SAND WORKS release keystore (never legacy; never committed).
3. Sign, verify, and distribute the APK privately (sideload/private channel). Document version + release notes.

Architecture — SAND WORKS (online-first)

Platform

• **App:** Android native, **Kotlin + Jetpack Compose**, Material 3.

• **Application id / package:** `com.roshan.sandworks`.

• **Backend: Firebase** — Authentication, Cloud Firestore (data), Cloud Storage (profile photos / exported files where enabled), Cloud Functions (server-authoritative operations), Cloud Messaging (notifications), App Check, Crashlytics.

• **Model: online-first.** Firestore is the authoritative store. Devices read from and write to the live backend through security rules. This is a deliberate contrast to offline-first: there is no local-primary store and no offline queue acting as the source of truth.

Layer overview (clean architecture)

1. **UI layer (Compose):** screens + view models. One screen per destination from `docs/screens/`. UI shows truthful states and never fabricates success.

2. **Domain layer:** pure business logic — money engine maths, distribution rules, eligibility, trip rules — independent of Firebase. Unit-testable.

3. **Data layer:** repository contracts implemented against the backend (Firestore reads; Cloud Functions for privileged writes). Also non-privileged local caching only for UX (e.g. remembering session), never as a correctness source.

Server-authoritative operations (Cloud Functions)

The following are never computed or authorised only on the client — the backend is authoritative:

• Trip numbering (collision-safe).

• Owner provisioning / role grant.

• Driver & labourer approval.

• Rate & money-rule snapshot at record time.

• Money distribution and daily accrual.

• Daily closure (per organisation+date, idempotent).

• Weekly/monthly leaderboards.

• Temporary-assignment expiry enforcement.

• Alert send and notifications.

• Export (PDF/CSV) and profile-photo handling (where storage enabled).

• Audit log creation.

Concurrency & integrity

• Optimistic concurrency with a revision/version on mutable records; on conflict the user is shown the situation and chooses — never a silent last-writer-wins overwrite.

• Idempotency keys on server-authoritative operations so retries never double-apply (critical for daily closure and money).

• Money is integer currency (no floating point).

Connectivity reality

• Online-first means screens show **loading / error / offline-with-retry** honestly when the network is unavailable; the user is never given a fake "saved" state.

• See `docs/quality/` for the expected behaviour under flaky networks.

Non-functional targets

• Release APK target ≈ 15–25 MB (acceptable ≤ 50 MB); never trade security/reliability/a11y/integrity for size.

• Fast, reliable, accessible, secure — industry-level quality bar.

DATABASE — Firestore Data Architecture — SAND WORKS

Status: **SPECIFIED.** Cloud Firestore is the authoritative backend. Model is online-first. Money in integer paise. Backend-authoritative fields are never client-written.

Conventions

• Money fields stored as integer paise.

• All writes from Cloud Functions / rules-authorised; mutable docs carry `revision` (optimistic concurrency); idempotency keys on server ops.

• Server timestamps for createdAt/updatedAt.

• Org scoping: every doc path includes orgId.

Collections & documents

`orgs/{orgId}`

Fields: name; ownerUid; settings { defaultRatePaise=20000, distributionRule, summaryTime (default "19:30", tz Asia/Kolkata), retentionPrefs }; createdAt/updatedAt. Server-authoritative settings writes.

`users/{uid}`

Fields: orgId; name; role (OWNER/DRIVER/LABOURER); phone; email; status (pending/active/rejected/disabled/suspended); approvedBy/approvedAt; profilePhotoUrl?; createdAt/updatedAt; revision. Role/status are server-authoritative (Cloud Functions). OWNER exactly one.

`tractors/{id}`

Fields per tractors.md: orgId; name; model; identifier?; active; createdAt/updatedAt; revision. Indexes: orgId+active, orgId+name.

`trips/{id}`

Fields: orgId; date; time; tractorId; driverUid; labourerUids[]; labourerPresence[]; rateSnapshotPaise (immutable); totalPaise; tripNumber (server, collision-safe); revision; voidedAt?/voidedBy?/voidReason?; createdAt/updatedAt; idemKey. Indexes: orgId+date, orgId+tractorId+date, orgId+driverUid+date, orgId+labourerUid+date (array-contains for labourer). TripNumber unique per org.

`attendance/{orgId}_{labourerUid}_{date}`

Fields per attendance.md: orgId; labourerUid; date; state; source; correctedBy?; reason?; revision; timestamps.

`rates/{id}` (history) and trip snapshots

• `orgs/{orgId}` current defaultRate; each trip stores immutable `rateSnapshotPaise`. A `rateChanges` sub-collection/log may record rate history (audit). Never recompute history.

`distributionRules/{id}` / org settings

Current rule in org settings; per-trip/closure uses snapshot or current at closure as documented.

`closures/{orgId}_{date}`

Fields: orgId; date; status; tripIds[]; perUserAccruals map {uid: paise}; summary; generatedBy; timestamps; idemKey; audit. Idempotent boundary (org,date).

`earnings/accruals` (per user per period)

Derived/append-only from closures: `accruals/{orgId}_{periodKey}` or subcollection per user. Owner/person read per role.

`leaderboards/{orgId}_{week|month}_{period}`

top3 computed by backend; honest ranks; deterministic tie.

`tempAssignments/{id}`

Per temporary-access.md fields incl. start/expiry/status/scope/reason; backend-enforced expiry; indexes orgId+targetUser+status.

`notifications/{id}`

recipientUid; type; payload; readAt?; sentAt; deepLink target.

`messages/{id}` and `alerts/{id}`

Owner-only broadcast. message/alert: orgId, senderUid, recipients[], message, ackBy[], sentAt, deliveryState, history.

`exports/{id}`

orgId; requester; range; breakdown; format; status; fileRef; createdAt.

`audit/{id}`

orgId; actorUid; action; targetType; targetId; before/after; timestamp. **Server-written only.**

`appConfig` / org config

Version/gates as needed (server).

Integrity & consistency

• Transactions for multi-doc money writes (closure). Batch where atomic.

• Idempotency keys prevent double closure/duplicate.

• Query rules: rules are not filters — queries must match rules (documented query/rule matrix).

• Soft-delete/deactivation for tractors/users (status/active) — never hard-delete history.

• No cross-org/cross-user reads.

Query & index plan

Documented composite indexes needed (Firestore): trips by (org,date); (org,tractor,date); (org,driver,date); (org,labourer via array-contains,date); users by (org,role,status); notifications by (recipient,unread); audit by (org,actor,date); closures by (org,date); leaderboards by (org,period). Each deployed with rules.

API / Backend Operations — SAND WORKS

Status: **SPECIFIED.** Logical API contract. Implemented via Firebase Cloud Functions (callable) + Firestore Security Rules. Backend-authoritative.

For every operation we specify: name, caller roles, input, validation, authorization, server behaviour, output, errors, idempotency, audit, retry, transaction requirements.

Operations

auth-provision-owner

Caller: provisioning (operator/CLI). Behaviour: create org + owner (Ramesh Sahu) + set role OWNER. Idempotent by email/uid. Audit.

register-user (driver/labourer)

Caller: self (via Sign Up). Input: name, role request (DRIVER/LABOURER), phone, email. Validation. Behaviour: create pending user; notify owner. Output: pending. Audit.

approve-user / reject-user / set-user-status

Caller: OWNER. Input: target uid, decision. Authorization: owner. Behaviour: set status active/rejected/disabled/suspended; notify (C); audit. Idempotent.

create-temp-assignment / revoke-temp-assignment

Caller: OWNER (authorized DRIVER for create within limits). Input: labourer uid, start, expiry, scope, reason. Backend enforces expiry (server time). Audit. Revoke by owner.

create-trip / edit-trip / void-trip

Caller: DRIVER (own), OWNER (any). Input fields. Authorization: role + own scope + active tractor/labourers + approval. Behaviour: assign trip number (collision-safe), snapshot rate, compute totalPaise, revision. Idempotent by idemKey (no duplicate). Audit. Transaction.

configure-rate

Caller: OWNER. Set defaultRatePaise. Validation integer>0. Future trips snapshot new; history unchanged. Audit.

configure-distribution

Caller: OWNER. Set rule (equal/driver+labour/custom %/fixed). Validate sums. Audit.

run-daily-closure(date, org)

Caller: scheduled/trigger or owner fallback. Behaviour: snapshot eligible trips, compute per-user accrued (integer paise), write closures/{org}_{date} idempotently (exactly-once), notify daily accrued-money summary to eligible, audit. Idempotency: same (org,date) → no double count. Transaction. Honest fallback documented if no scheduled server path.

compute-leaderboard(period)

Backend at week/month boundary. top-3 honest; deterministic tie; audit.

mark-attendance / correct-attendance

Caller: OWNER (correct requires reason). Revision/conflict. Audit.

send-alert / send-message (owner broadcast)

Caller: OWNER. Input: message, recipients. Behaviour: authorize, FCM deliver strongest-compliant urgent (alert) / normal (message), record delivery state/history/audit. Duplicate prevention. Never claims silent/DND override.

export-data

Caller: OWNER. Generate PDF/CSV (server-side where plan supports; honest fallback otherwise). Audit.

profile-photo operations

Self/owner. Validate/compress; Storage rules (plan-dependent). Audit.

audit-read

Caller: OWNER, read-only.

Error contract (common)

auth/permission-denied, not-found, validation, conflict(revision), duplicate, rate-limited, timeout, backend-unavailable, feature-unavailable(plan), session-expired, assignment-expired. See ERROR-STATES.md.

Firestore Security Rules summary

• users: owner may write status/role via CF only; user reads own.

• trips: driver may create/own-edit; owner any; labourer no write.

• closures/accruals/leaderboards/audit/alerts: server/CF write; reads by role/org.

• attendance: owner write.

• tractors: owner write; active readable by org D/L as authorized.

• org settings: owner write.

Rules enforce the matrix; queries match rules (rules are not filters).

SECURITY — SAND WORKS

Status: **SPECIFIED.** Backend-enforced; never rely on UI restrictions as security.

1. Identity & auth

• Firebase Authentication (email/password). Session management; sign-out; password reset via backend.

• App Check enabled to gate backend access to the genuine app.

• Approved users only get operational access; approval backend-enforced.

2. Authorization & Firestore Security Rules

• Role enforcement (OWNER/DRIVER/LABOURER), owner-only operations, driver authorization (own scope), labourer read-only enforcement.

• Org scoping + user scoping at the rule level. No cross-org/cross-user private-money reads.

• Temporary access enforced with server time + expiry; no permanent escalation.

3. Server authority

• Money, trip numbering, approval, closure, leaderboards, audit, expiry, alert/message delivery are Cloud-Functions-authoritative; rules deny client writes to those fields.

4. Session, least privilege, data isolation

• Least privilege per role matrix; data isolation per org/user.

• Disabled/suspended/expired session truthfully denies.

5. Input & output validation

• Validate on server (not just client). Output-filtered reads (rules). No injection.

6. Abuse prevention

• Rate limiting on callable functions; duplicate-submission protection (idempotency keys + in-flight guard); replay protection for idempotent ops; notification-abuse prevention (owner-only send; per-recipient). Export protection (owner-only). Profile-image security (rules + size/MIME/dim).

7. Secrets & transport

• Secrets never committed (tokens, service keys, signing). TLS/HTTPS enforced by Firebase. Android network security config appropriate for the domain; certificate handling standard.

8. Logging & auditability

• Audit events server-written; append-only; owner read. No client audit writes. No PII in analytics.

9. Privacy & retention

• Private/family data; retention window per org prefs; delete/export per documented policy.

10. Incident handling

• Owner + maintainers process; documented in DEPLOYMENT.md (rollback, monitoring).

11. Dependency & device security

• Dependency scanning in CI; secure Android storage (EncryptedSharedPreferences/Keystore for tokens where needed); release signing dedicated SAND WORKS keystore.

Attack posture test list (mapped to TESTING)

Role escalation, cross-org by id change, forging role/approval/rate/number/closure/audit/timestamp, disabled/expired access, deep-link authz, storage abuse, idempotency/retry duplication, notification forgery/broadcast of another's money.

ERROR / LOADING / EMPTY / OFFLINE STATES — SAND WORKS

Status: **SPECIFIED.**

Loading

• Initial loading: skeleton/spinner where appropriate.

• Button progress during submits; **prevent duplicate submission** (disable while in flight).

• Never show fake "loading completed" then fake success.

Empty

Honest empty states with context + action (usually reset/new):

• no trips, no users, no pending approvals, no tractors, no labourers, no leaderboard participants, no accrual, no notifications, no search results, no export history.

• Empty states never fabricate data or default values that mislead.

Error (typed catalogue)

• authentication failure; permission denied; network failure; Firebase/backend unavailable; timeout; rate limited; validation error; conflict (concurrent edit/revision); duplicate request; expired session; expired temporary access; feature unavailable (plan); server error; export failure; notification failure; image upload failure.

Each: message, optional retry, stays on screen (no fake nav), no data loss of in-progress form.

Offline / network degradation (online-first)

• Connection indicator when offline.

• Retry; exponential backoff where appropriate (sync/retry tasks).

• Stale cached display allowed ONLY clearly marked as not authoritative/stale.

• **Never show "saved successfully" unless the backend accepted the authoritative write.**

• Failed write → pending/error + retry; duplicate prevention; recovery after reconnection re-syncs correctly.

Success

• Real confirmation after backend acceptance; snackbar/state as appropriate.

Special account states

• Permission denied / Forbidden, Unauthorized/SessionExpired, AccountBlocked, AssignmentExpired, NetworkUnavailable, MaintenanceUnavailable.

Global state layer

Referenced by SCREEN-STATE.md; each screen maps its operations to these states.

PERFORMANCE & Stability — SAND WORKS

Status: **SPECIFIED.** Measurable targets; regression criteria in TESTING/CI.

Targets

• Cold start: ≤ ~2–3 s on a mid-range device (screen first frame).

• Navigation: screen transition responsive (< ~300 ms perceived).

• Screen rendering: 60 fps target, no jank from list/image churn.

• Firestore reads: bounded per screen; paginated where large; no unbounded full-collection pulls.

• Query size: paginate trips/history (e.g. page size documented, e.g. 25–50); avoid loading all notifications.

• Image loading: Coil; thumbnails; downsampled; no giant master images loaded in UI.

• Notification processing: token refresh handled; no heavy main-thread work.

• Memory: no unbounded caches; release bitmap memory; list recycling via LazyColumn.

• Battery/network: minimal polling; pull-to-refresh not auto-loop; push-driven updates; WorkManager only where justified.

• APK size: **target 15–25 MB; practical upper boundary 50 MB**. Never remove important functionality solely for a number; do not add bloat.

Monitoring & regression

• Baseline timing recorded in CI (debug/perf).

• Performance regression gate: if a change increases cold-start/scroll jank beyond threshold, fail review.

• Firestore usage reviewed for read-cost and rule-query compliance.

• Crash/ANR via Crashlytics (when provisioned) monitored.

Observability

• Crashlytics (plan/credentials) for stability; minimal analytics (no PII) only if justified. See DEPLOYMENT/quality docs.

Notifications & Emergency Warning — SAND WORKS

Status: **SPECIFIED.** FCM-based, online. Backend-authoritative. Honest about Android limits.

1. Types

2. Normal notifications

Approve account/access-change/trip/daily-accrual-summary/operational/system events → targeted per user, sent by backend. No private-financial cross-broadcast (never include another user's money).

3. Owner-only broadcast & Emergency Warning

• Only OWNER can send broadcast messages/warnings.

• Emergency Warning flow: **warning button → confirmation step → optional message → recipient scope → delivery state → retry → duplicate prevention → history → audit → cancellation where possible.**

4. Urgent behaviour — strongest Android-compliant (honest)

Use a **high-importance notification channel**, with:

• **vibration** (when device permits)

• **notification sound / alert sound** (when device/OS permits)

• **heads-up notification** (where the OS allows)

• **full-screen intent only where legally/platform appropriate** (restricted).

Accurately distinguish: vibration · notification sound · heads-up · high-importance channel · full-screen intent · device volume · Silent mode · Do Not Disturb · Android permission/policy restrictions.

The app never claims it can force a phone to play at full volume while Silent/DND, bypass Do Not Disturb, or override the user's sound settings. Document the cases where Android prevents the app from overriding system policy; the app provides the strongest compliant behaviour and tells the user to enable the channel/permissions for best effect.

5. Read state, retention, channel model

• Notification centre with read/unread; badge. Deep link → re-validate then route.

• Channels: Earnings/Summary, Alerts/Warnings, System/Account, Operational.

• Retention per org prefs (history).

6. Permission handling

• POST_NOTIFICATIONS requested with rationale + in-app settings; no over-broad upfront demand. If denied, features degrade honestly (user may enable later).

7. Plan dependency

Real FCM push requires Firebase project + FCM creds (owner-provisioned). Not fabricated; document fallback if absent (in-app notification centre still works; real push pending).

Core Workflows — SAND WORKS

1. Account creation & approval (online)

1. New user opens SC-AUTH-WELCOME → SC-AUTH-SIGNUP, selects role (driver or labourer) and enters details.
2. Backend creates a **pending** account (role selection is a request only).
3. Owner sees it in SC-OWN-APPROVALS and approves/rejects.
4. On approval the account is activated; the user signs in and lands on their role dashboard. Rejected users see an honest rejection state.
5. (Owner account is provisioned by the operator — the one-owner rule.)

2. Daily operation — a trip

1. Driver opens SC-DRV-DASH and taps **+ ADD TRIP**.
2. Driver picks date/time, tractor (from active registry), labourers present; rate snapshot auto-applies; backend assigns a collision-safe trip number.
3. On save, backend authorises (role, own-scope, tractor/labourer valid, approval active), snapshots the rate, and records the trip. Any conflict is surfaced, not silently overwritten.
4. The trip now counts toward attendance, accrual and leaderboards.

3. Daily closure / accrual (server-authoritative)

1. At the configured time (default 19:30), the backend produces one daily closure for the organisation+date.
2. It snapshots eligible trips, applies the distribution rule per trip, and computes each user's accrued totals.
3. Closure is idempotent — re-running never double-counts.
4. Eligible users are notified with accrued-totals wording.

4. Attendance corrections (owner)

1. Owner marks a day working/absent, or **corrects** an existing record with a required reason.
2. Correction is audited; no silent overwrite. A concurrent change surfaces a conflict.

5. Reports & export (owner)

1. Owner builds a report over a date range and breakdown (overall/per tractor/per driver/per labourer/per date).
2. Owner exports PDF or CSV. Backend enforces owner-only scope. No fabricated totals.

6. Temporary assignment & expiry (labourer in a driver-like role)

1. Owner (or an authorised driver within limits) creates a temporary assignment with start/end/reason/scope.
2. The backend enforces expiry: access ends automatically at `end`; the assignee cannot extend it.
3. Expiry may trigger a type-E notification.

7. Operational alert (owner)

1. Owner composes a message and picks recipients.
2. Backend delivers a high-priority, prominent, acknowledgeable alert to those users (honest platform limits apply).
3. Acknowledgement is recorded.

8. Notifications → deep link

1. A notification arrives for a user.

2. Opening it re-validates auth + role + org + ownership + existence, then routes to the right screen; otherwise an honest Forbidden/NotFound is shown.

ACCESSIBILITY — SAND WORKS

Status: **SPECIFIED.**

Requirements

• **Screen reader labels:** all screens, controls, list items, icons have contentDescriptions/semantics.

• **Semantic hierarchy:** headings/roles announced in order.

• **Touch targets:** ≥48dp (with 8dp spacing).

• **Contrast:** AA minimum text; non-text contrast; both themes.

• **Text scaling:** layouts support large font scale without truncation/overlap.

• **Keyboard navigation:** logical tab/focus order where applicable; IME actions.

• **Content descriptions / focus order:** correct; focus visible.

• **Error announcements:** errors announced to screen readers (live region).

• **Dynamic font sizes:** supported.

• **Reduced motion:** honour system reduce-motion (limit animation).

• **Haptics alternatives:** not the only feedback; visual/verbal alternatives.

• **Colour-independent status:** working/absent, success/error, leaderboard ranks also conveyed by icon/text, not colour only.

Acceptance

Automated (Compose semantics/contrast) + manual TalkBack pass across all screens; empty/error/offline states readable. Screen reader can complete core flows (sign-in, view accrual, add trip for driver/owner).

SEO — SAND WORKS (honest assessment)

Status: **SPECIFIED (documented, not implemented).**

Applicability

SAND WORKS is a **private Android application** (package `com.roshan.sandworks`) for single-owner personal/family use. It is **not a public website** and is not intended for Google Play publication. Therefore:

• Traditional public-web SEO (meta tags, sitemaps, backlinks, keyword ranking) is **not a core requirement** and must not be fabricated as such.

• This document exists to state that explicitly, so a future agent does not invent meaningless SEO work.

What applies

• **Android app presence only:** if ever listed anywhere private/alpha (internal test), set accurate store-listing metadata (name "SAND WORKS", short description, category) — that is "app store listing", not web SEO.

• **App Indexing:** only relevant if the app exposes public web content — it does not. Not applicable.

• **Future website/landing page (PROPOSED, not approved):** if the owner later wants a public landing page, then and only then would conventional SEO/metadata apply. Not part of this build.

Non-goal guardrail

Do not add webpages, meta-tags, or public-indexable content for a private APK. Do not report "SEO implemented." This is intentionally documented as out of scope until the owner requests a website.

TESTING — SAND WORKS

Status: **SPECIFIED.** Comprehensive. Covers unit → end-to-end. No test disabled to make CI green.

Layers & what is tested

• Unit: money engine (integer paise, distribution, rounding remainder, rate snapshot immutability, duplicate detection), domain rules, time/day/timezone boundaries, tie-break determinism.

• ViewModel: state machine per screen, UDF action→reaction, no fake success, submission-guard.

• Repository/data: mapping, org/user scoping.

• Firestore Security Rules tests (emulator): role escalation, cross-org, forge role/approval/rate/number/closure/audit/timestamp, disabled/expired, deep-link authz, storage, idempotency/retry.

• Backend function tests (emulator): register/approve, create/void trip + numbering, configure rate/distribution, daily closure idempotency (run twice → no double count), leaderboard (top-3 honest), temp-assignment expiry, alert/message delivery, export, attendance correction.

• Notifications tests: permission, token lifecycle, deep-link re-validation, no cross-user money broadcast, honest Android limits.

• Navigation tests: role routing, bottom-nav, back/top-left arrow, session-expiry/assignment-expiry routing, deep-link authz.

• Compose UI tests: each screen + states (loading/empty/error/offline/submitting/conflict/forbidden), interactions real (no dead buttons).

• Accessibility tests: semantics, contrast, touch targets, reduced motion, colour-independent status, TalkBack core flows.

• Offline/network degradation: retry, no fake success, failed-write handling, recovery/re-sync, duplicate prevention.

• Concurrency: revision conflict surfaced (no silent overwrite).

• Retry/idempotency: closure, trip create.

• Temp-access expiry, export (authorization + correctness), profile-image (validation/storage), end-to-end (sign-up→approval→trip→closure→summary), regression, security suite, performance.

Acceptance criteria (representative)

• Money: for given trips, computed accruals equal expected integer values (property tests + examples); Σ shares == pool.

• Closure: calling closure(org,date) twice yields identical single accrual; leaderboard never shows fabricated 3rd when <3 qualify.

• Security suite: all attack-posture cases denied.

• Rules: non-owner writes rejected; labourer writes rejected; cross-org reads rejected.

• Every role's core journey passes end-to-end on emulator.

Tooling (recommended, not fabricated as running)

• JUnit/truth/mockito/mockk; Robolectric; Firebase Emulator Suite (rules + functions); Compose UI testing; Gradle managed devices/emulator where CI supports.

CI / CD — SAND WORKS

Status: **SPECIFIED.** Continuous integration; **no automatic Google Play publishing**. Private APK distribution.

Pipeline stages

1. **Checkout** (pin commit).
2. **Dependency resolution** (cached Gradle).
3. **Formatting/static checks** (ktlint/spotless).
4. **Lint** (Android lint; no errors).
5. **Compilation** (`assembleDebug`/`compileDebugKotlin`).
6. **Unit tests** (JVM + Robolectric).
7. **Firebase emulator tests** (Security Rules + Cloud Functions) where secrets-free (uses emulator; no production project).
8. **UI tests** where feasible (Gradle managed devices / connectedAndroidTest on hosted runner, if available).
9. **Security scanning** (SAST).
10. **Secret scanning** (gitleaks/detect-secrets) — fail on any token/key.
11. **Dependency scanning** (OWASP/OSV) — fail on critical/high.
12. **Artifact generation** — build release APK unsigned (or signed in secure step).
13. **Versioning** — derived from git/tag; versionCode monotonic.
14. **Release validation** — tests green + manual checklist.
15. **Signing** — only in protected release job via CI secrets; never in logs.
16. **Artifact integrity** — checksum + signature verify.
17. **Release notes** — generated/manual; attached.
18. **Distribution** — private APK output (no Play).

Guardrails

• No secrets in pipeline logs; env/CI secrets only.

• Never disable tests to go green.

• Placeholder scan (TODO/FIXME/TBD/XXX) fails the production branch job.

• Release signing uses dedicated SAND WORKS keystore from secrets; not committed.

Current state

Pipelines are **SPECIFIED**; actual CI runner/config is EXISTING only when committed and run. This repo is documentation; no CI is running here.

DEPLOYMENT & FIREBASE PLAN DEPENDENCIES — SAND WORKS

Status: **SPECIFIED.** Clearly separates environments and plan dependencies. No real secrets included.

Environments

• **Local development:** Android Studio, debug signing, local emulator or a dedicated dev Firebase project.

• **Firebase Emulator Suite:** Firestore/Auth/Functions emulation for tests — no production data.

• **Test environment:** a separate Firebase project (owner-provided) for integration.

• **Production Firebase:** owner-provided project with Auth, Firestore, App Check, FCM, Crashlytics (+ Storage/Cloud Functions/Analytics per plan).

• **Private APK release:** signed with the dedicated SAND WORKS release keystore; distributed privately (sideload/private channel).

Firebase project setup (owner-provided inputs)

• Create Firebase project; enable Authentication (email/password); Firestore; App Check (Play Integrity); Cloud Messaging; Crashlytics; Analytics (only if justified); Storage and Cloud Functions only if the plan supports them.

• Provide `google-services.json`/config via secure env (never committed).

Plan dependencies (do not assume free-plan availability)

Documented as environment/release dependencies; the implementation stops at that boundary (AGENT.md §9).

Deployment steps (production)

1. Apply Firestore Security Rules + indexes.
2. Deploy Cloud Functions (as plan supports).
3. Configure Authentication, App Check, FCM.
4. Verify rules/functions via emulator then production smoke tests (seed a test user only).
5. Set environment separation; no cross-env data.
6. Build + sign release APK (dedicated keystore from secrets); verify integrity; install on device.
7. Rollback: revert functions via prior deployment; APK rollback = reinstall prior signed APK (same key).
8. Backups: Firestore export/scheduled backup per plan; keep org data retrievable.
9. Disaster recovery: documented restore procedure.
10. Monitoring: Crashlytics + (optional) minimal analytics; function logs.

Guardrails

• No real secrets in docs/config.

• Production vs test separation.

• Owner retains Firebase ownership/access.

Implementation Control Plane — SAND WORKS

Status: **SPECIFIED.** Small, independently verifiable tasks. An independent senior Kotlin+Jetpack Compose coding agent builds task-by-task in dependency order. No giant "build the entire app" task.

Task contract template (every task carries)

ID · objective · requirements · dependencies · affected screens · affected data · backend requirements · security requirements · acceptance criteria · test requirements · completion evidence · blockers · next task/dependency.

Phases (26) — see PHASES.md

1 Project foundation · 2 Design system & assets · 3 Authentication · 4 User/role/approval system · 5 Firestore data layer · 6 Backend/security rules · 7 Tractor management · 8 Trip management · 9 Money/accrual engine · 10 Attendance · 11 OWNER experience · 12 DRIVER experience · 13 LABOURER experience · 14 Notifications · 15 Emergency warning · 16 Messaging · 17 Leaderboards · 18 Search/filter/sort · 19 Profile/photo · 20 Export · 21 Error/network resilience · 22 Observability · 23 Security hardening · 24 Performance · 25 Complete testing · 26 Release preparation.

References

• DEPENDENCY-GRAPH.md, ROADMAP.md, TRACEABILITY-MATRIX.md, STATUS.md.

Status classification & blockers

See STATUS.md. Items requiring owner/Firebase inputs are recorded, not fabricated. Do not claim DONE without completion evidence + tests.

PHASES & TASKS — SAND WORKS Implementation Control Plane

Status: **SPECIFIED.** Each phase lists its small tasks (P<n>T<m>). Every task is fully defined per the template in README.md. Dependencies, roadmap, traceability and status are in the sibling files. Task status is NOT-STARTED by default and flips only on completion evidence (see STATUS.md).

Phase 1 — Project foundation

• P1T1 Scaffold Android project, applicationId `com.roshan.sandworks`, min/compile/target, Kotlin + Compose + Material 3, version catalogs. Blockers: none (buildable now).

• P1T2 Hilt DI foundation; Application; modules.

• P1T3 Navigation Compose skeleton + Auth gate + role routing stubs (real later).

• P1T4 Configuration & secrets handling scaffold (no secrets committed; env-injected).

• P1T5 Base logging + typed error infra.

Phase 2 — Design system & assets

• P2T1 Apply locked brand assets (logo/app icon masters → launcher). No redraw/SVG.

• P2T2 Theme tokens (light+dark) per DESIGN-SYSTEM.md; typography, spacing.

• P2T3 Component library (buttons, cards, forms, dialogs, sheets, chips, badges, lists, avatars, loading/empty/error visuals).

• P2T4 Responsive + accessibility baseline (semantics, contrast, touch targets, reduced motion).

Phase 3 — Authentication

• P3T1 Firebase Auth integration (email/password) behind repository; emulator/local.

• P3T2 Session management; token/credential; sign-out.

• P3T3 Auth screens (Welcome, Sign In, Sign Up, Forgot, reset) + validation + states.

• P3T4 Role-aware routing on auth state; deep-link re-validation foundation.

• Blockers: real Firebase project/App Check for production (emulator fine).

Phase 4 — User / role / approval system

• P4T1 User model + pending/active states.

• P4T2 Registration (driver/labourer) → pending; owner provisioning path.

• P4T3 Approvals UI + Approve/Reject backend ops.

• P4T4 Disable/suspend/reactivate; account status screens (Approval Pending/Rejected/Suspended/Blocked).

• Security: owner-only; backend-enforced; audit. No admin role. Mansingh Rana = DRIVER.

Phase 5 — Firestore data layer

• P5T1 Repositories (org, users, tractors, trips, attendance, closures, notifications, messages, audit).

• P5T2 DTOs/field mapping; integer paise money type; server-authoritative field guards.

• P5T3 Revision-based optimistic concurrency helpers; idempotency keys.

• P5T4 Pagination/query helpers per DATABASE.md indexes.

Phase 6 — Backend / security rules

• P6T1 Firestore Security Rules (role × operation × org/own scope) per roles-access + SECURITY.

• P6T2 Cloud Functions: register/approve/status, trip create/number/void, rate & distribution config, closure, leaderboard, attendance, temp assignment, alert/message, export, profile-photo (plan-dependent).

• P6T3 App Check + rules test suite (emulator).

• Blockers: real Firebase + Blaze for CF/Storage where required.

Phase 7 — Tractor management (OWNER)

• P7T1 Tractor list/add/edit/deactivate/reactivate + validation/duplicate.

• P7T2 Tractor detail: per-tractor totals + history.

• Security: owner write; active readable for trip use.

Phase 8 — Trip management

• P8T1 Add/Edit trip (owner/driver own), rate snapshot, labourer select.

• P8T2 Backend trip number + validation; duplicate/conflict handling.

• P8T3 Trip history + filters (date, driver, labourer, tractor, status).

• Security: driver own / owner any; labourer read-only.

Phase 9 — Money / accrual engine

• P9T1 Money domain (integer paise, distribution, rounding remainder, snapshot) unit tests.

• P9T2 Daily accrued-money closure (idempotent org+date) via function; honest fallback.

• P9T3 Accrual overview + person accrual detail; wording accrued-only.

Phase 10 — Attendance

• P10T1 Attendance mark/correct (owner, reason); derived working/absent.

• P10T2 Working/absent days/dates surfacing (labourer + owner).

Phase 11 — OWNER experience (shell + screens per OWNER.md)

• P11T1 Owner shell + bottom nav (Home/Trips/People/More) + drawer.

• P11T2 Dashboard, Daily Summary, Trip management.

• P11T3 Users/approvals/temp access screens.

• P11T4 Tractors, Rates, Money/Accrual, closure, summaries.

• P11T5 Attendance, leaderboards, emergency warning, messages.

• P11T6 Export center, audit, profile/settings.

Each: states/a11y/responsive/tests.

Phase 12 — DRIVER experience (per DRIVER.md)

• P12T1 Driver shell + bottom nav; dashboard.

• P12T2 Add/Edit trip + tractor/labourer selection; own history.

• P12T3 Today's trips, daily total, accrued money, share today's trips.

• P12T4 Driver profile/settings/notifications; temporary-assignment status.

Phase 13 — LABOURER experience (per LABOURER.md)

• P13T1 Labourer shell + bottom nav; dashboard (metrics).

• P13T2 My days/history, accrued money, remaining.

• P13T3 Leaderboards (weekly/monthly top-3 + own rank).

• P13T4 Labourer profile/settings/notifications/account status. Read-only enforced.

Phase 14 — Notifications (centre + infra)

• P14T1 FCM token lifecycle + permission (rationale) + channels.

• P14T2 Notification centre + types A–F + deep links + read state.

• Security: per-user targeting; no cross-user money broadcast.

• Blockers: FCM project creds for real push.

Phase 15 — Emergency warning

• P15T1 Owner warning composer + confirmation; strongest-compliant urgent delivery; delivery state/history/retry/duplicate prevention/audit/cancel where possible. Honest Android limits; no silent/DND-override claims.

Phase 16 — Messaging (owner broadcast)

• P16T1 Owner message composer/history; targeted delivery; audit.

Phase 17 — Leaderboards

• P17T1 Weekly + monthly top-3 backend computation + honest rank rule.

• P17T2 Leaderboard UI (owner/driver/labourer) per leaderboards.md.

Phase 18 — Search / filter / sort

• P18T1 Filter/sort UI primitives (chips, date range, search w/ debounce) per search-filter-sort.md.

• P18T2 Apply to trips, users, tractors, notifications, exports, accrual.

Phase 19 — Profile / photo

• P19T1 Profile + edit (own).

• P19T2 Profile photo upload/replace/delete + validation/compression (plan-dependent; honest if unavailable).

Phase 20 — Export

• P20T1 Owner export center (range/breakdown/format), server-side where plan supports, history, share. Honest fallback.

Phase 21 — Error / network resilience

• P21T1 Global error/offline/retry layers per ERROR-STATES.md + SCREEN-STATE.md.

• P21T2 Failed-write handling + recovery; no fake success.

Phase 22 — Observability

• P22T1 Crashlytics integration (when provisioned); minimal no-PII analytics (only if justified).

Phase 23 — Security hardening

• P23T1 Attack-suite tests; rules/functions emulator security tests; secrets hygiene audit; App Check verify.

Phase 24 — Performance

• P24T1 Apply PERFORMANCE targets; pagination, image downsampling, list recycling; measure.

Phase 25 — Complete testing

• P25T1 Full matrix per TESTING.md; accessibility; e2e; security; concurrency; retry/idempotency. No test disabled to pass.

Phase 26 — Release preparation

• P26T1 Build/sign release with dedicated keystore (from CI secrets).

• P26T2 Release validation checklist; APK integrity; private distribution.

• P26T3 Handover + release notes.

• Blockers: release keystore (owner), Firebase project/plan.

Phase 1 — Project Foundation — Task Contracts

Status: SPECIFIED. Tasks buildable now (no Firebase needed). Blockers: none for local build (release signing is Phase 26).

P1T1 Scaffold project

Objective: buildable Kotlin/Compose/M3 Gradle project, applicationId `com.roshan.sandworks`.

Requirements: app module, version catalog, Material3, min/target SDK per plan; package never `com.roshan.labourparty`.

Affected screens: none (scaffold). Data: none. Backend: none. Security: no secrets; signing later.

Acceptance: `assembleDebug` builds; package correct. Tests: build. Evidence: build log. Next: P1T2.

P1T2 Hilt DI foundation

Objective: DI graph (app/domain/data). Acceptance: compile + DI unit. Next: P1T3.

P1T3 Navigation skeleton + Auth gate

Objective: Navigation Compose graph, auth gate, role-route stubs (real in P3/P4). Acceptance: routes compile; back deterministic. Next: P1T4.

P1T4 Config & secrets scaffold

Objective: secrets/config from env/ignored (google-services dropped in later); nothing committed. Next: P1T5.

P1T5 Logging + typed errors

Objective: logger + sealed typed errors → message mapping (SCREEN-STATE). Next: Phase 2.

Phases 2–4 — Task Contracts

Phase 2 — Design system & assets

P2T1 Apply locked brand (master → launcher; no redraw/SVG). Obj: correct launcher/app icon from masters. Accept: icon shows; no placeholder. Evidence: screenshot/asset check.

P2T2 Theme tokens light+dark per DESIGN-SYSTEM. Accept: contrast AA; both themes.

P2T3 Component library. Accept: components used, no inert controls.

P2T4 Responsive + accessibility baseline (semantics, touch ≥48dp, reduced motion).

Phase 3 — Authentication

P3T1 Firebase Auth integration behind repository (emulator/local). Obj: email/password auth. Accept: sign-in/up/out works on emulator; never fake auth. Security: real Firebase creds in Phase 26/deploy only.

P3T2 Session management (persist, expiry, sign-out). Accept: session states truthful.

P3T3 Auth screens + validation + states (Welcome/SignIn/SignUp/Forgot/Reset). Accept: SCREEN-STATE + a11y; tests.

P3T4 Role-aware routing + deep-link re-validation foundation. Accept: routes by role from session.

Blockers (production): Firebase project + App Check.

Phase 4 — User / role / approval

P4T1 User model + pending/active/status. Accept: states model.

P4T2 Registration (D/L)→pending; owner provisioning path (Ramesh Sahu OWNER; no create-owner UI). Security: role = request only.

P4T3 Approvals UI + Approve/Reject backend op (CF). Owner-only. Accept: approved user gains access; audit.

P4T4 Disable/suspend/reactivate + account-status screens. Accept: no access when disabled; honest.

Security: no ADMIN role; Mansingh Rana = DRIVER. Enforced by backend not UI.

Phases 5–9 — Task Contracts (data layer, rules, tractors, trips, money)

Phase 5 — Firestore data layer

P5T1 Repositories (org/users/tractors/trips/attendance/closures/notifications/messages/audit) per DATABASE.md. Accept: repo contracts compile; emulator read/write.

P5T2 DTOs + integer-paise Money + server-authoritative field guards. Accept: no float for money.

P5T3 Revision concurrency + idempotency helpers. Accept: conflict surfaced; idempotent ops.

P5T4 Pagination/query helpers matching indexes/rules (rules are not filters).

Phase 6 — Backend / security rules

P6T1 Firestore Security Rules per roles-access/SECURITY. Accept: rules tests green (escalation, cross-org, forge, expiry, storage, idempotency).

P6T2 Cloud Functions ops (register/approve/status; trip create/number/void; rate & distribution; closure; leaderboard; attendance; temp assignment; alert/message; export; profile-photo). Accept: emulator function tests per API.md.

P6T3 App Check + emulator security suite.

Blockers: real Firebase + Blaze for CF/Storage.

Phase 7 — Tractor management (OWNER)

P7T1 Tractor list/add/edit/deactivate/reactivate + validation/duplicate. Accept: owner CRUD; duplicate rejected.

P7T2 Tractor detail: per-tractor totals + history. Accept: derived from trips; inactive keeps history.

Phase 8 — Trip management

P8T1 Add/Edit trip (driver own / owner any): date/time/tractor/labourers/rate snapshot/total. Accept: driver own + owner any; labourer no write.

P8T2 Backend number + validation + duplicate/conflict. Accept: number collision-safe; duplicate rejected; conflict surfaced.

P8T3 Trip history + filters. Accept: search/filter/sort per spec; paginated.

Phase 9 — Money / accrual engine

P9T1 Money domain unit tests: integer paise, distribution (equal default + config), rounding remainder (Σ==pool), snapshot immutability. Accept: property/examples pass.

P9T2 Daily accrued-money closure idempotent (org,date) via function; honest fallback. Accept: re-run → no double count; wording accrued.

P9T3 Accrual overview + person detail. Owner; own-scope for D/L. Accept: no cross-user money.

Phases 10–13 — Task Contracts (attendance, OWNER, DRIVER, LABOURER experience)

Phase 10 — Attendance

P10T1 Attendance mark/correct (owner, reason), derived working/absent, revision conflict. Accept: corrections audited; no silent overwrite.

P10T2 Working/absent days & dates surfaced (labourer + owner). Accept: counts correct per attendance.md.

Phase 11 — OWNER experience (per OWNER.md; screens must satisfy SCREEN-STATE, a11y, responsive, tests)

P11T1 Shell + bottom nav (Home/Trips/People/More) + drawer.

P11T2 Dashboard, Daily Summary, Trip management (add/edit/detail/history/search).

P11T3 Users list/detail; Approvals/Pending; Temporary Access.

P11T4 Tractors; Rates; Money/Accrual overview + person detail; Daily closure; Weekly/Monthly summaries.

P11T5 Attendance; Leaderboards; Emergency warning; Messages.

P11T6 Export center; Audit; Owner profile; Settings; Security/account.

Security: every capability owner-only + validation/authz/audit (no arbitrary powers).

Phase 12 — DRIVER experience (per DRIVER.md)

P12T1 Shell + bottom nav (Home/My Trips/My Totals/More).

P12T2 Dashboard (+ADD TRIP), tractor/labourer selection.

P12T3 Add/Edit trip (own), backend number, own history.

P12T4 Today's trips, daily total, accrued money, share today's trips (real values).

P12T5 Driver profile/settings/notifications; temporary-assignment status.

Security: own scope; driver never admin; no access to others' money.

Phase 13 — LABOURER experience (per LABOURER.md; read-only)

P13T1 Shell + bottom nav (Home/My Days/Leaderboard/More).

P13T2 Dashboard metrics (trips, accrued, remaining, working/absent days & dates).

P13T3 My days/history/date detail; accrued money; read-only trip history.

P13T4 Weekly/monthly top-3 + own rank (honest).

P13T5 Profile/settings/notifications/account-status.

Security: read-only enforced; no write actions exposed/authorised.

Phases 14–20 — Task Contracts (notifications, warning, messaging, leaderboards, search, profile, export)

Phase 14 — Notifications

P14T1 FCM token lifecycle + permission (rationale, in-app settings) + channels (Earnings/Summary, Alerts, System, Operational).

P14T2 Notification centre + types A–F + deep links (re-validated) + read state.

Security: per-user; no cross-user money broadcast; backend send.

Blockers: real FCM creds for real push; in-app centre works regardless.

Phase 15 — Emergency warning

P15T1 Owner warning composer + confirmation + optional message + recipients + strongest-compliant urgent delivery (high-importance channel, sound, vibration, heads-up where OS permits; never silent/DND override) + delivery state/history/retry/duplicate prevention/audit/cancel where possible. Accept: honest limits documented; no fabricated "full volume".

Phase 16 — Messaging (owner broadcast)

P16T1 Owner message composer/history; targeted recipients; delivery + audit. Owner-only.

Phase 17 — Leaderboards

P17T1 Weekly + monthly top-3 backend computation + honest rank (no fabricated ranks when <3 qualify). Accept: idempotent; deterministic tie; period Asia/Kolkata.

P17T2 Leaderboard UI (owner/org, driver own+top, labourer top-3+own).

Phase 18 — Search / filter / sort

P18T1 UI primitives (chips, date range, search w/ debounce, sort, reset, empty state, pagination) per search-filter-sort.md.

P18T2 Apply to trips, users, tractors, notifications, accrual/export history. Accept: permissions-respecting; rules-compliant queries.

Phase 19 — Profile / photo

P19T1 Profile + edit (own). Accept: own record only.

P19T2 Profile photo upload/replace/delete + validation/compression/crop + progress/failure/retry. Blockers: plan-dependent (Storage/Blaze) — mark unavailable honestly if plan lacks it.

Phase 20 — Export

P20T1 Owner export center (range/breakdown/format PDF/CSV), server-side where plan supports, history, share; honest fallback. Security: owner-only.

Phases 21–26 — Task Contracts (resilience, observability, security, perf, testing, release)

Phase 21 — Error / network resilience

P21T1 Global error/offline/retry/connection-state layers per ERROR-STATES.md + SCREEN-STATE.md. Accept: no fake success; failed writes handled; recovery after reconnect.

P21T2 Retry w/ exponential backoff where appropriate; duplicate prevention on reconnect.

Phase 22 — Observability

P22T1 Crashlytics integration (when provisioned). Optional minimal no-PII analytics only if justified.

Blockers: project/credentials (deploy-time).

Phase 23 — Security hardening

P23T1 Attack-suite tests (escalation, cross-org, forge, expiry, deep-link, storage, idempotency) + secrets hygiene audit + App Check verification. Accept: all cases denied.

Phase 24 — Performance

P24T1 Apply PERFORMANCE targets (pagination, image downsampling, LazyColumn, no unbounded reads); measure cold-start/scroll; APK size 15–25MB/≤50MB.

Phase 25 — Complete testing

P25T1 Full test matrix per TESTING.md (unit/VM/repo/rules/functions/UI/a11y/network/concurrency/retry/e2e/security/perf). Accept: criteria met; no test disabled to pass.

Phase 26 — Release preparation

P26T1 Build/sign release with dedicated SAND WORKS keystore (CI secrets; never committed).

P26T2 Release validation checklist (security retest on release candidate; a11y; money-idempotency; audit integrity); APK integrity.

P26T3 Private distribution + release notes + handover.

Blockers: release keystore (owner), Firebase project/plan.

DEPENDENCY-GRAPH — SAND WORKS Implementation

Status: SPECIFIED.

```
P1 Foundation
 ■■ P2 Design system/assets
 ■■ P3 Authentication ■■
P5 Firestore data      ■■ P4 User/role/approval
 ■■ P6 Backend/rules   ■
P7 Tractors ■■
P8 Trips     ■■ P9 Money/accrual engine
P10 Attendance■
P11 OWNER  ■■ (depend on P1-10)
P12 DRIVER  ■■■ P14 Notifications ■ P15 Emergency warning ■ P16 Messaging
P13 LABOURER■    ■
P17 Leaderboards (needs P9 trips data)
P18 Search/filter/sort (applies to P11/12/13 lists)
P19 Profile/photo
P20 Export (needs P9)
P21 Error/network resilience (cross-cutting; retrofits all screens)
P22 Observability ■ P23 Security hardening ■ P24 Performance ■ P25 Complete testing ■ P26 Release
```

Critical path

P1 → P2 → P3/P5/P6 → P4 → P7/P8/P10 → P9 → (P11■P12■P13) → P14 → P15/P16 → P17 → P18/P19/P20 → P21 → P22 → P23 → P24 → P25 → P26.

Parallelisable

P7, P8, P10 after P5/P6; P11/P12/P13 after P9; P17/P18/P19/P20 after their data deps; P21 retrofits across screens.

External deps (not tasks)

Firebase project (real) gates P3/P4/P6/P14/P15/P16/P19/P20/P22/P26 production verification; Blaze gates CF/Storage features; release keystore gates P26. See STATUS.md.

ROADMAP — SAND WORKS Implementation

Status: SPECIFIED. Milestones vs phases.

Milestone M1 — Foundations (Phases 1–2)

Project builds, design system + brand applied, DI/nav base. Non-cloud; buildable now. Exit: app compiles; brand + theme in place.

Milestone M2 — Identity & data (Phases 3–6)

Auth, user/approval, Firestore data layer, backend/rules. Emulator-testable; production needs Firebase project. Exit: sign-up→approval→role routing works on emulator; rules/function tests green.

Milestone M3 — Core domain (Phases 7–10)

Tractors, trips, money/accrual, attendance. Exit: trip→closure→accrual idempotent; money tests green.

Milestone M4 — Role surfaces (Phases 11–13)

OWNER, DRIVER, LABOURER experiences. Exit: each role's screens per SCREEN-CATALOG satisfy state/a11y/tests.

Milestone M5 — Engagement (Phases 14–20)

Notifications, emergency warning, messaging, leaderboards, search/filter/sort, profile/photo, export. Exit: owner can warn/message; leaderboards honest; notifications targeted.

Milestone M6 — Quality & release (Phases 21–26)

Resilience, observability, security, performance, testing, release. Exit: full matrix green; security suite green; release APK signed/verified.

Release readiness

Go-live (private APK) requires: Firebase project + plan (owner), release keystore (owner), approved copy/UX (owner) as applicable — recorded as blockers, not skipped.

TRACEABILITY-MATRIX — SAND WORKS

Status: SPECIFIED. Direction 1: requirement → feature → screen → data → backend → security → test → task. Direction 2: every task maps back to a real requirement. No orphan requirements/tasks.

Requirement → Feature → Screen → Task (representative, authoritative set)

Screen ↔ task coverage

Every screen in SCREEN-CATALOG.md is produced within the phase assigned (P11 OWNER, P12 DRIVER, P13 LABOURER, P3/P4 auth, P14 notifications, etc.). No screen is orphaned; no task lacks a requirement.

STATUS & BLOCKERS — SAND WORKS Implementation

Status: **SPECIFIED.** This is the source of truth for task state and classification. Uses AGENT.md §8 vocabulary: EXISTING / SPECIFIED / MISSING / BLOCKED / PROPOSED.

Application functionality

All application functionality is **SPECIFIED** (not implemented — this repo is documentation). No application code exists in this repository.

Implementation task status

All P<T> tasks are **NOT-STARTED** (planning) until a coding agent marks them IN-PROGRESS/DONE with completion evidence. Default NOT-STARTED. No task is fabricated as DONE.

Classification of areas

Blockers register (external inputs — do not fabricate)

Rule

An area depending on a BLOCKED item must be authored/emulator-tested to its boundary and reported, never faked. All other areas are fully specified and buildable to READY once Phase dependencies are met.